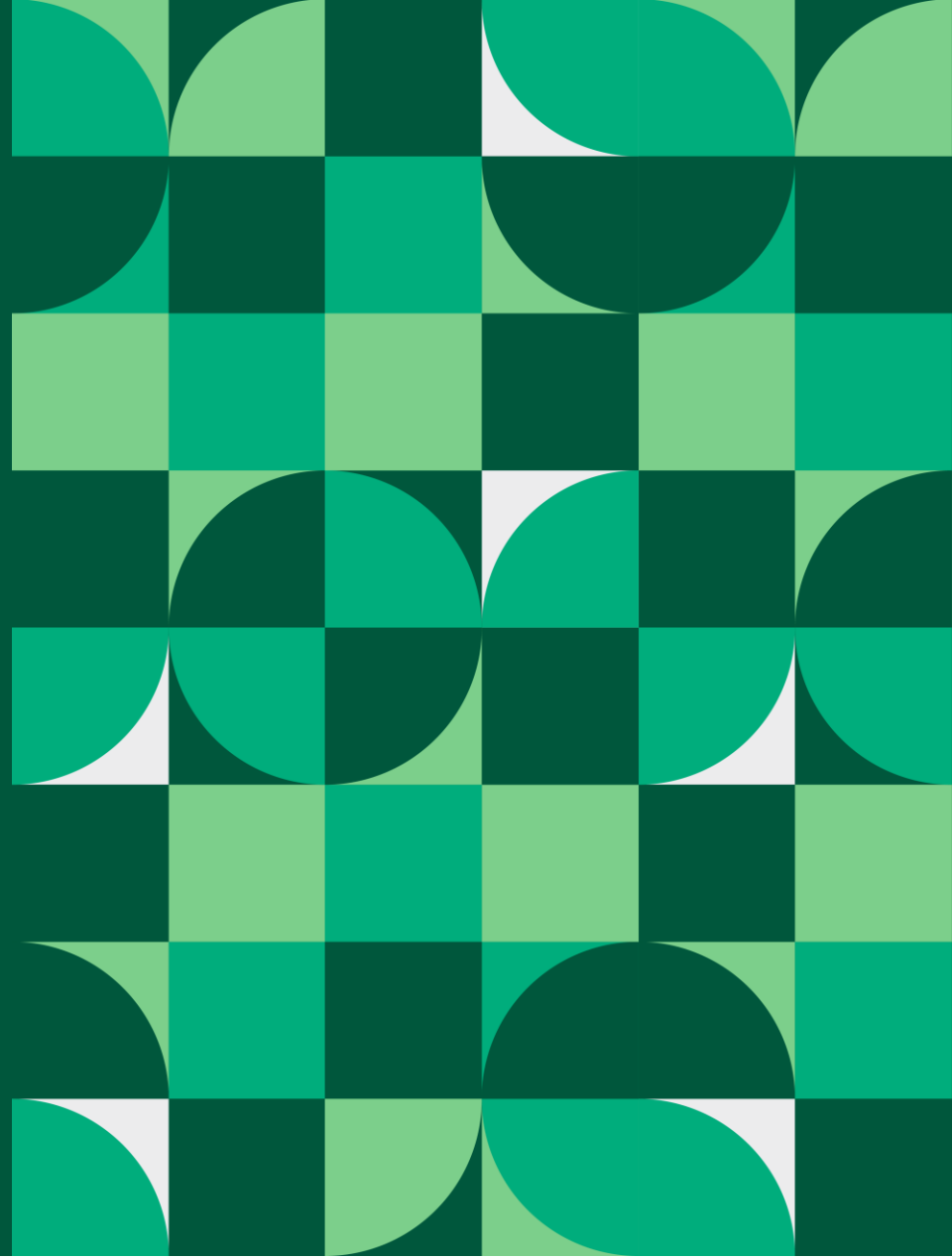
Signal Based Synchronization Fundamentals

Daniel Eaton

Chief Field Applications Engineer



EMERSON

Daniel Eaton
Chief Field Applications Engineer
CLA, CLED, 20 years “doing” w/ NI
Daniel.eaton@emerson.com

We are here to help!

- Pull us in to design conversations
- Let us help de-risk projects
- Tell us what you want to learn about next

Goals for Today

The Basics	Know Your Requirements → Pick Your Synchronization
Signal Based Synchronization	Common Vernacular
	Common Approaches
Signal Based Synchronization: The NI Way	The NI PXI Approach
	NI TCLK + NI PXI for Picosecond Synchronization

The Importance of Synchronization

EMERSON

Why?

Coordinate data signals relative to each other

Coordinate data signals relative to time

Poor synchronization leads to inaccurate results

When?

Data acquisition is being performed by multiple devices

- Multiple PXI modules in a single chassis
- Distributed across multiple PXI chassis

How Much Synchronization Do You Need?

Do you even have to care?

EMERSON

A Thermocouple vs RF

- 10 Hz vs 200 MHz carrier frequency
- Slow physical phenomenon → Nearly instant phenomenon
- 10s of microseconds → 10s of picoseconds
- With respect to time → with respect to other devices
- (my opinion) Eliminating drift is a must regardless of application

Know Your Requirements → Pick Your Path

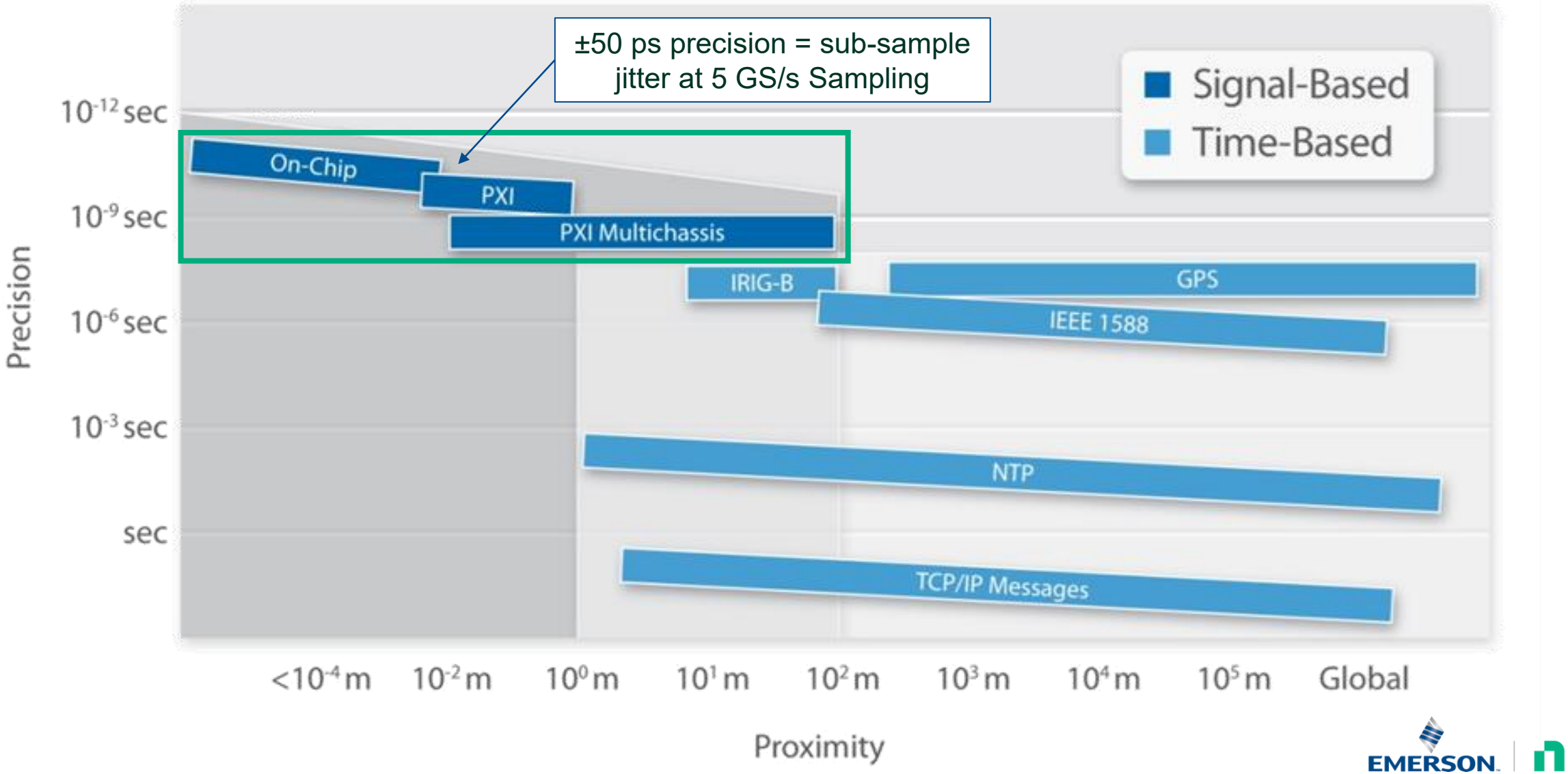
Signal-Based Synchronization

- No external time reference
- System is physically connected
- Offers the highest precision of synchronization
- Limited to cable distances of 200 m

Protocol-Based Synchronization

- Uses an external time reference
- IEEE 1588, GPS, and IRIG-B
- Useful for systems with a high number of nodes
- Nodes are connected through a network

Synchronization Methods Compared



If 1 microsecond is enough… NI TSN Enabled Devices…

TSN Synchronizes multiple devices with no programming required

Formalized as IEEE 802.1 AS (an IEEE 1588 profile)

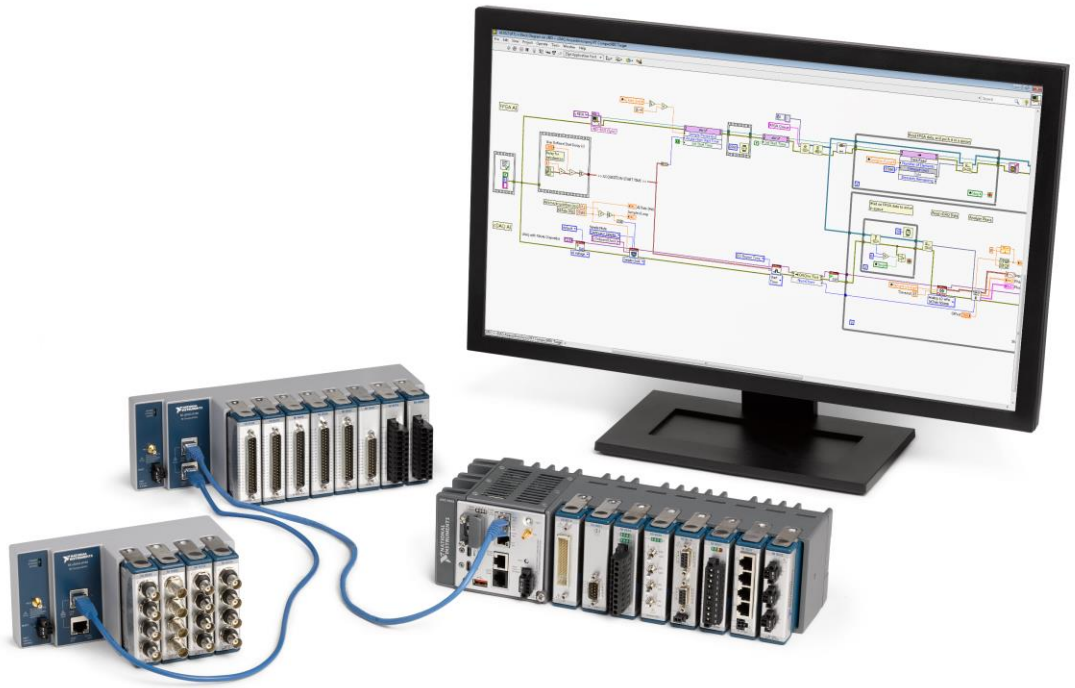
Synchronized networked systems within 1 µs of network time

Synchronize distributed measurements within 1 µs

- Same start time
- No drift over time

How: Plug in ethernet cables…

Supported Devices: TSN Enabled cRIO, cDAQ, & FieldDAQ



CALLBACK COMING…

EMERSON

Signal Based Synchronization

When to Use Signal-Based Synchronization

- Tight sample synchronization needs
  - Sub-nanosecond requirements
- Co-located hardware
  - Signal integrity diminishes over long distances
- Common sample clocking
  - Minimize device-to-device jitter and skew

Common Terms

Clocks

Clocks

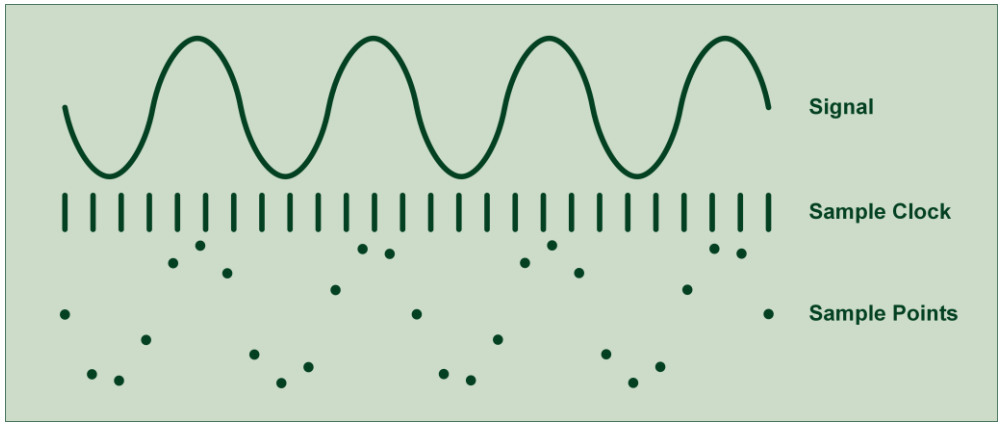
Periodic digital signals used to time the acquisition of data

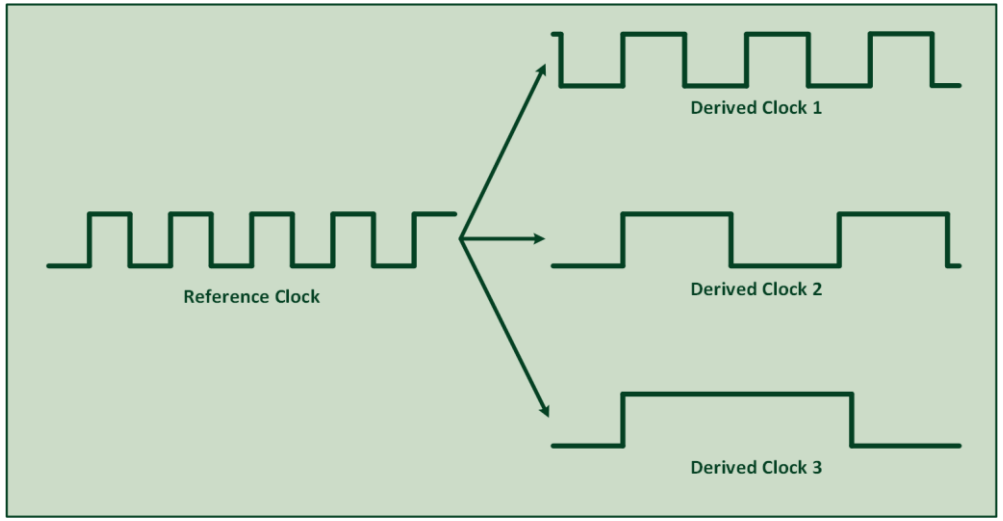
Sample Clock

Controls the timing of the removal samples from a data acquisition device

Reference Clock

A signal referenced by other systems' clocks to derive their own clock signals





Trigger

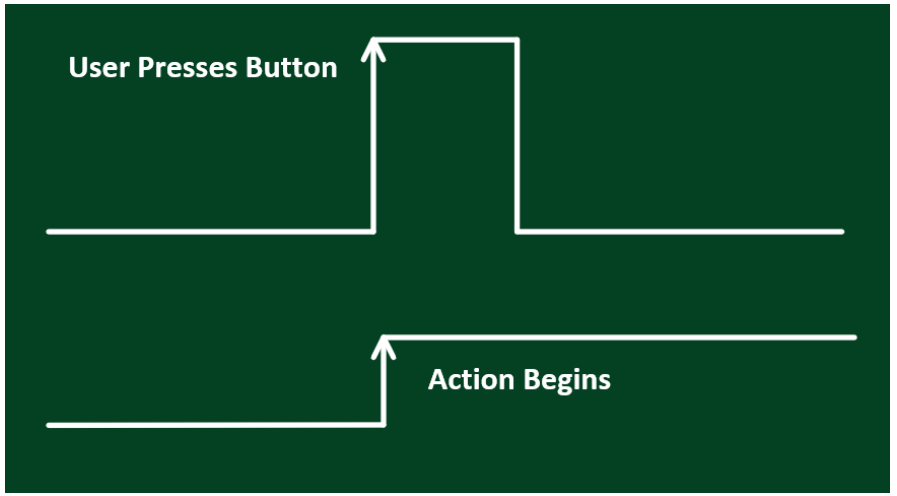
A trigger is a command that controls an acquisition.

Examples of Software Triggers

- Button press
- Alarm state
- External program

Examples of Hardware Triggers

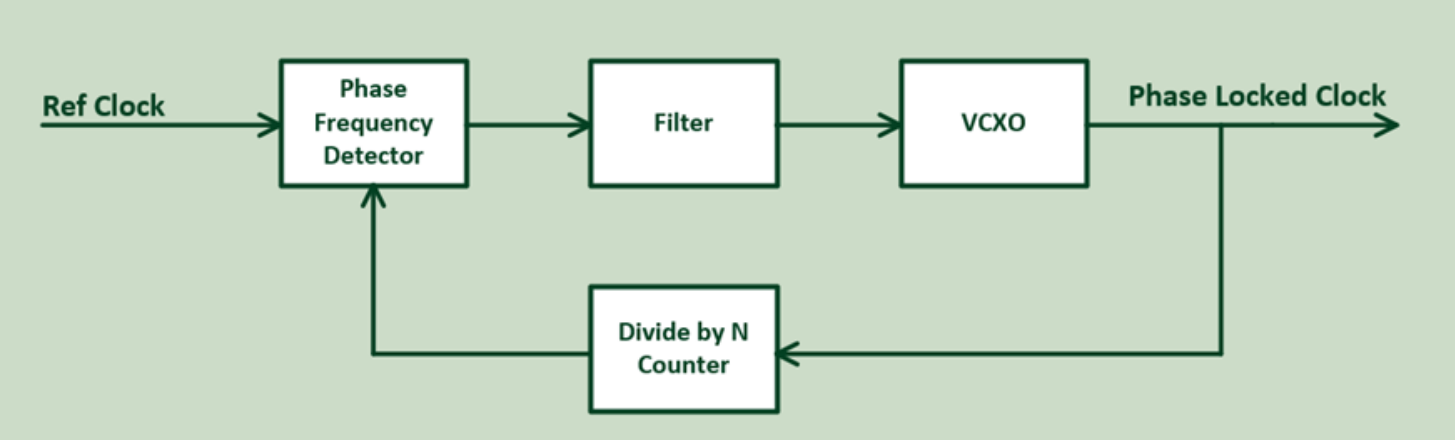
- PXI backplane triggers
- External Source (PFI Line, Digital Input)



Phase Locked Loop (PLL)

PLL is a type of feedback circuit that:

- Synchronizes phase of onboard clock with external timing source / reference clock
- Uses a Voltage Controlled Crystal Oscillator (VCXO)
- Allows multiple boards to lock to a shared reference signal

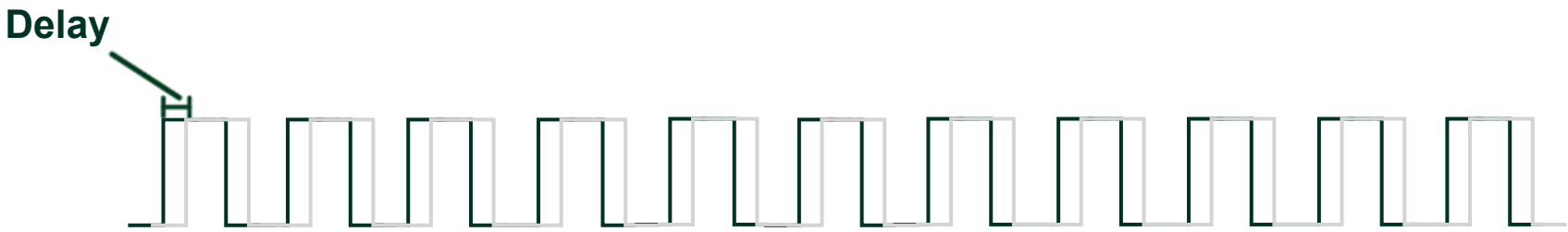


Common Challenges

Propagation Delay (component of skew)

Shifting of a clock or trigger by an offset in time

Roughly 1 ns/foot of cable



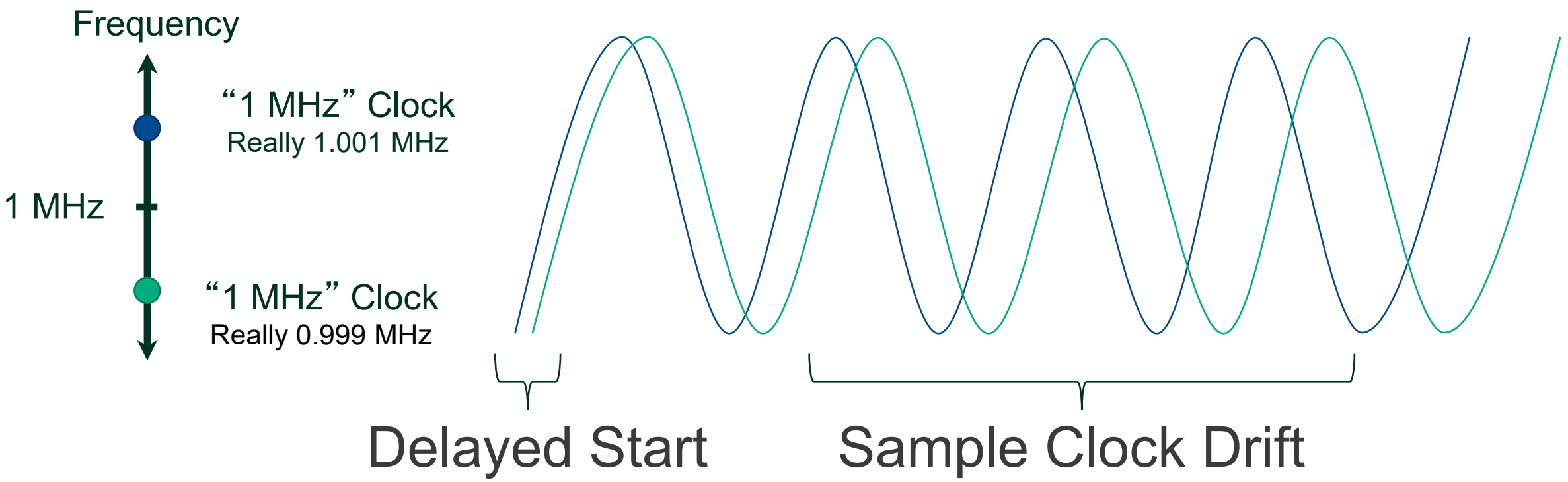
Drift

Occurs when two instruments are acquiring data at different sample rates

Error builds over time

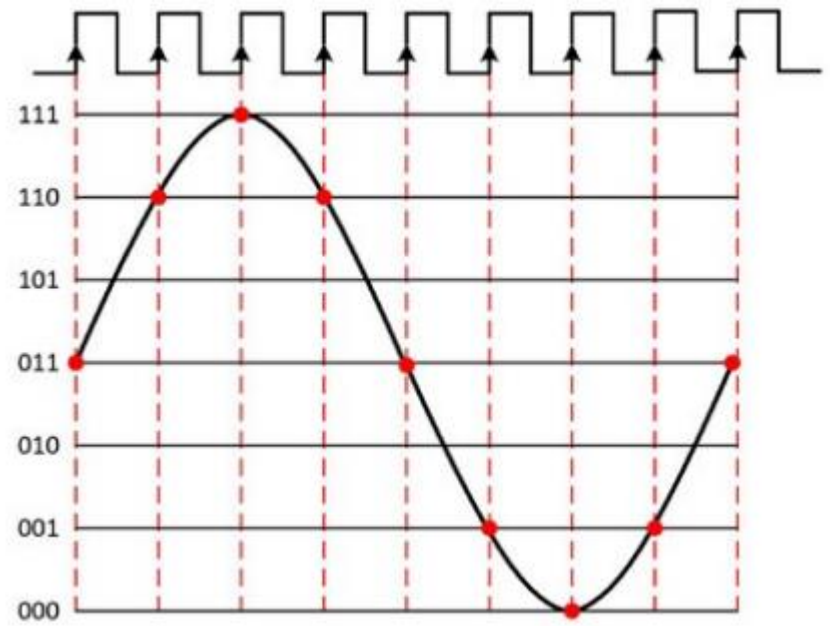
Clock Drift

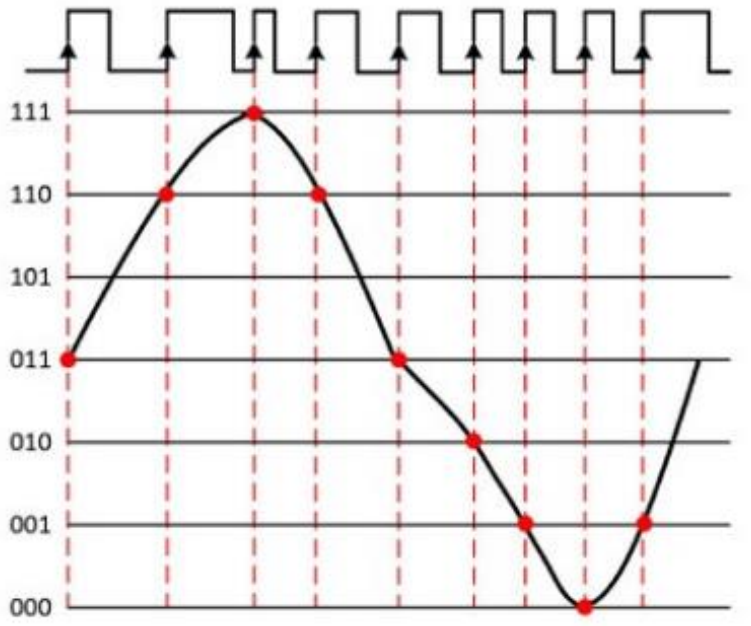
Errors grow as acquisition continues.



Jitter

Jitter is the deviation from the ideal timing of an event to the actual timing of an event.





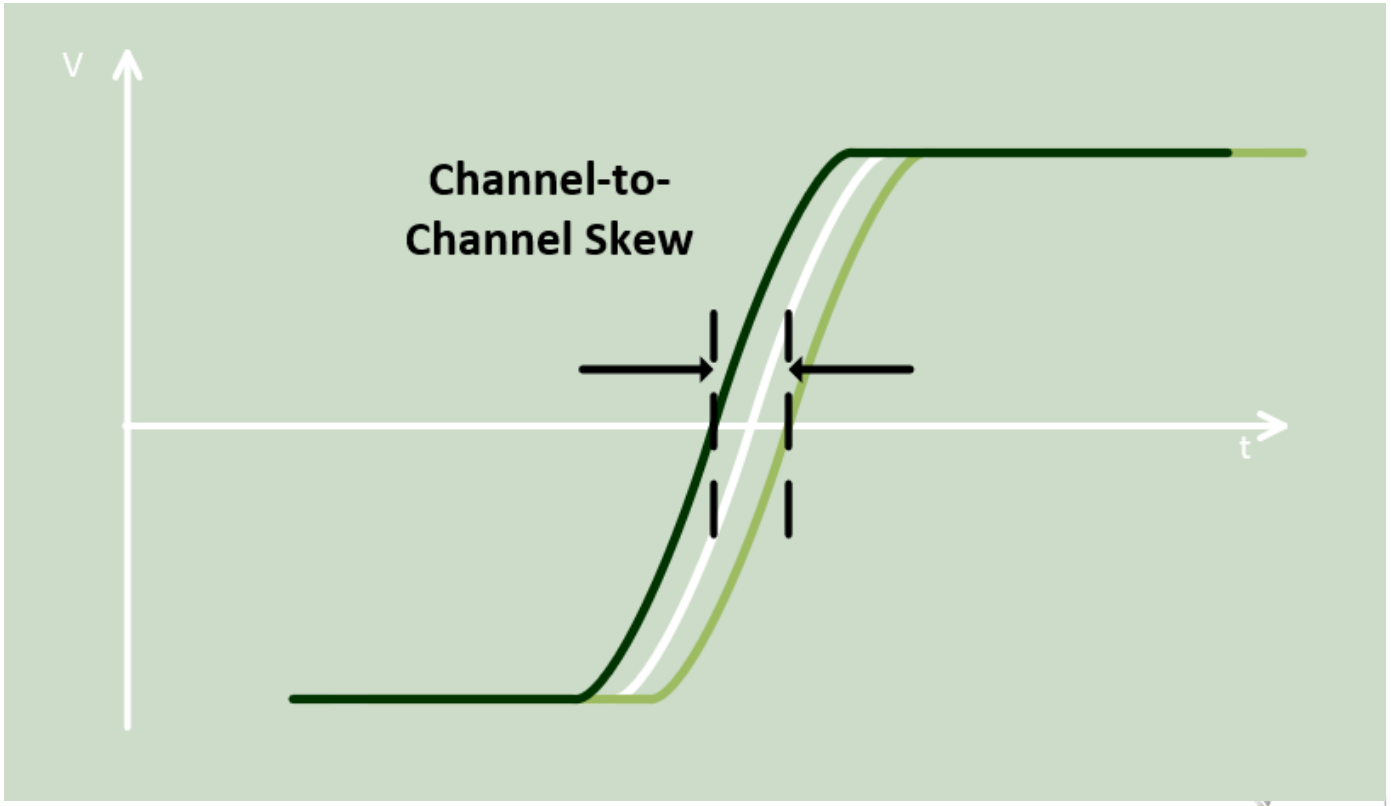
Skew

Skew is when the clock signal arrives at different components at different times.

Maintains clock period

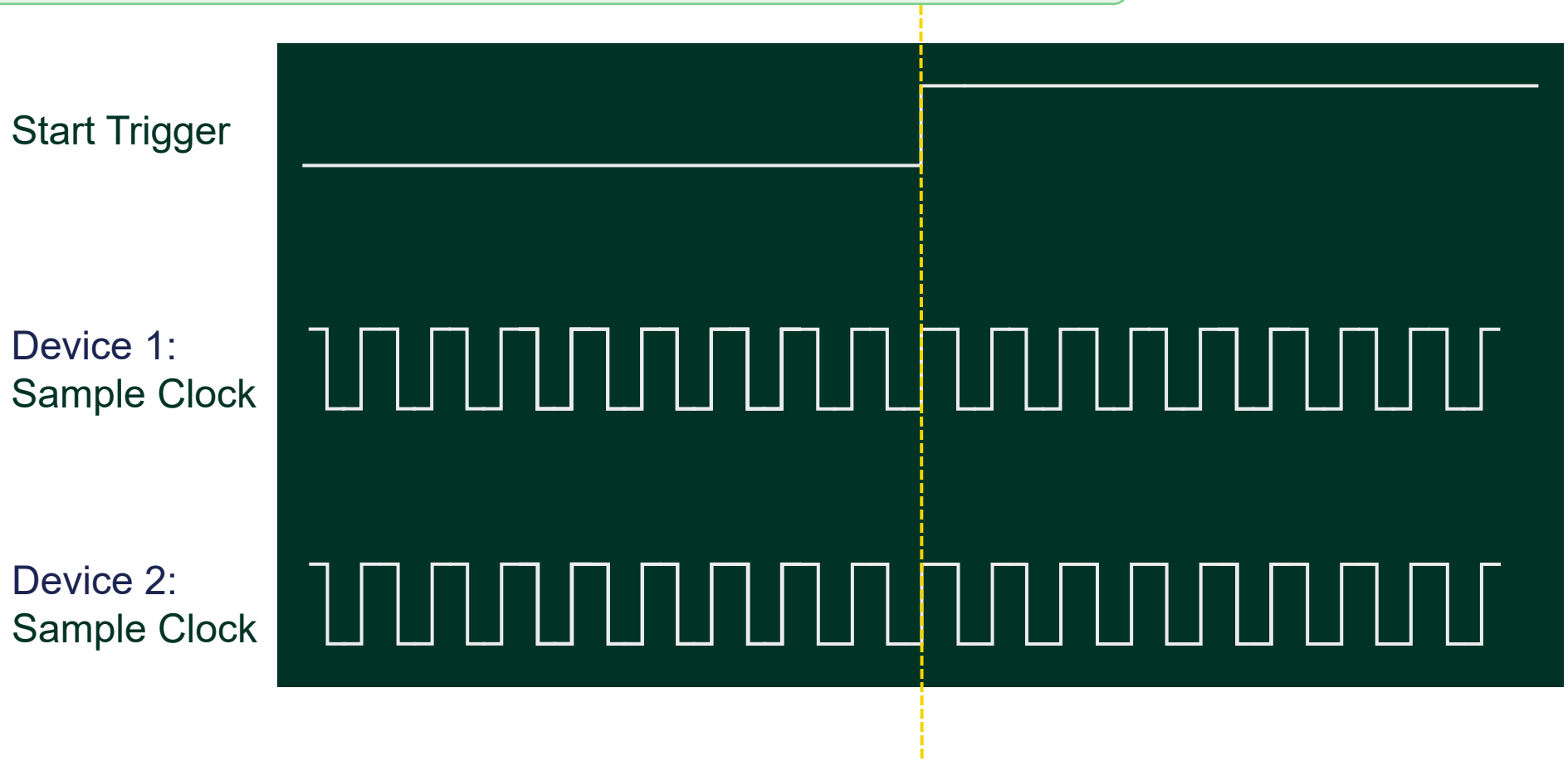
Causes:

- Wire Length
- Temperature Variation
- Different Input Capacitance



Edge Detection Inconsistencies

Distributing start time triggers and handling metastability



Common Methods

Demo System Setup

Device 1: MIO DAQ

- Generating Triggers / Clocks
- Generating Square Wave
- Looping back Square Wave

Wiring

- AI0: Reads Square Wave
- PFI4: Reads Start Trigger
- PFI0: Generates Square Wave
- PFI2: Generates Start Trigger; Clock

Device 2: cDAQ 9189 + 9215 (AI) + 9401 (DI)

- Read Triggers/Clocks
- Read Square Wave

Wiring

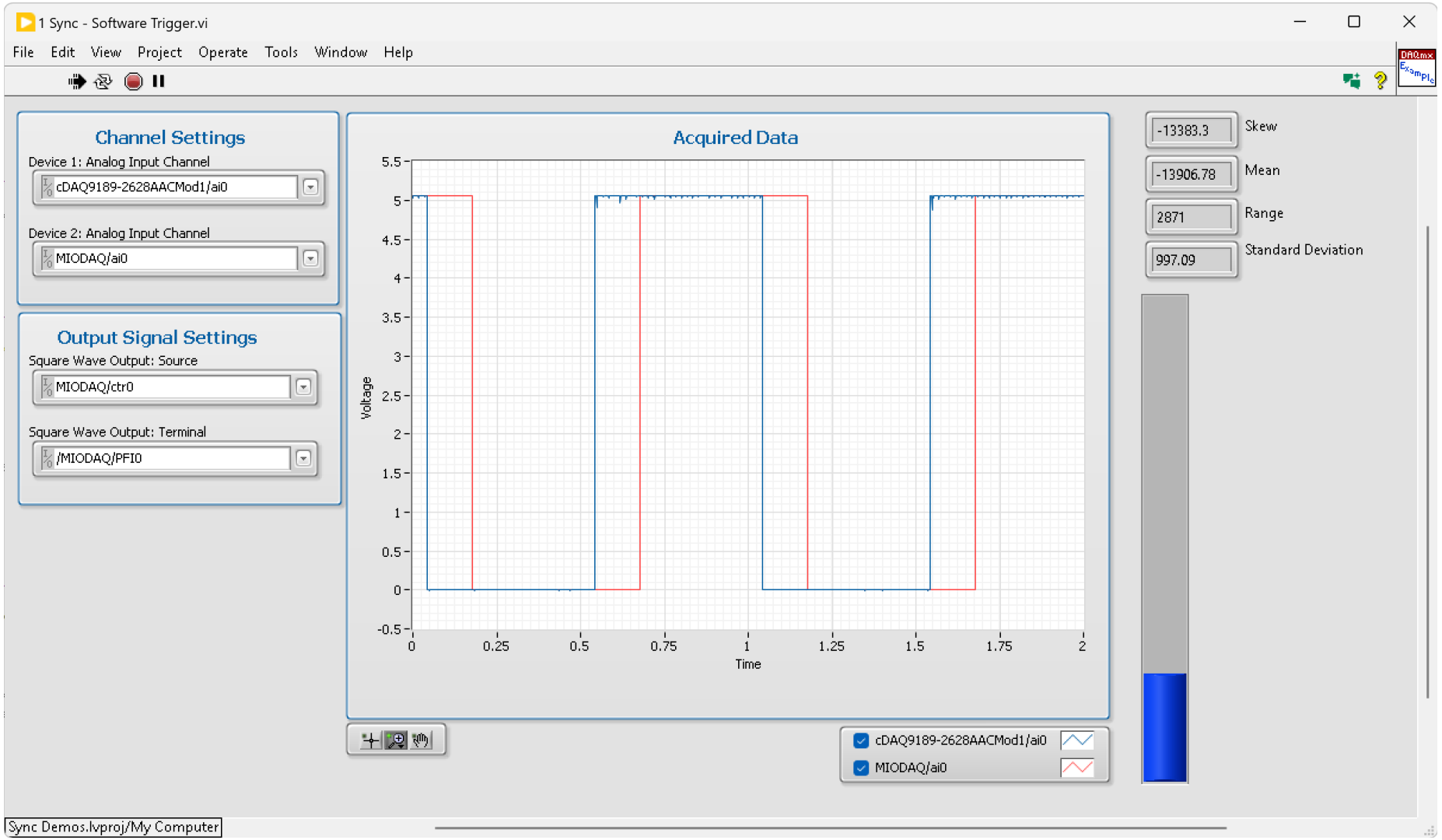
- 9215 AI0: Reads Square Wave
- 9401 DIO4: Reads Start Trigger, External Clock

Example Code Source

- https://github.com/eatondan/ni-LunchAndLearn-SignalBasedSynchronizationFundamentalsExamples
  - LabVIEW: Source\Challenges in Synchronization - NI DAQmx
  - Python: Source\Challenges in Synchronization - NI DAQmx - Python

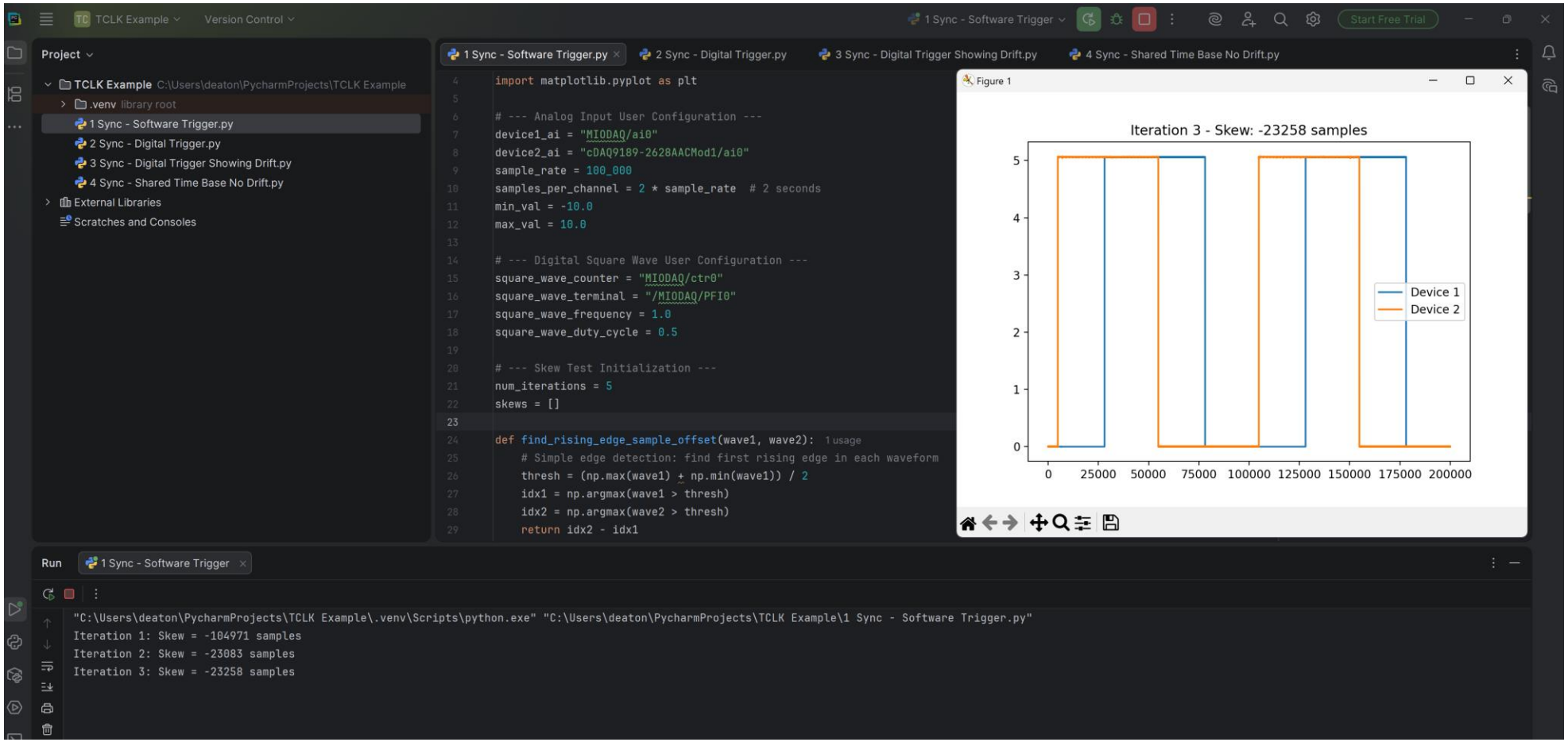
Demo 1: The Wrong Way…

- No Start Triggers
- Local Clocks



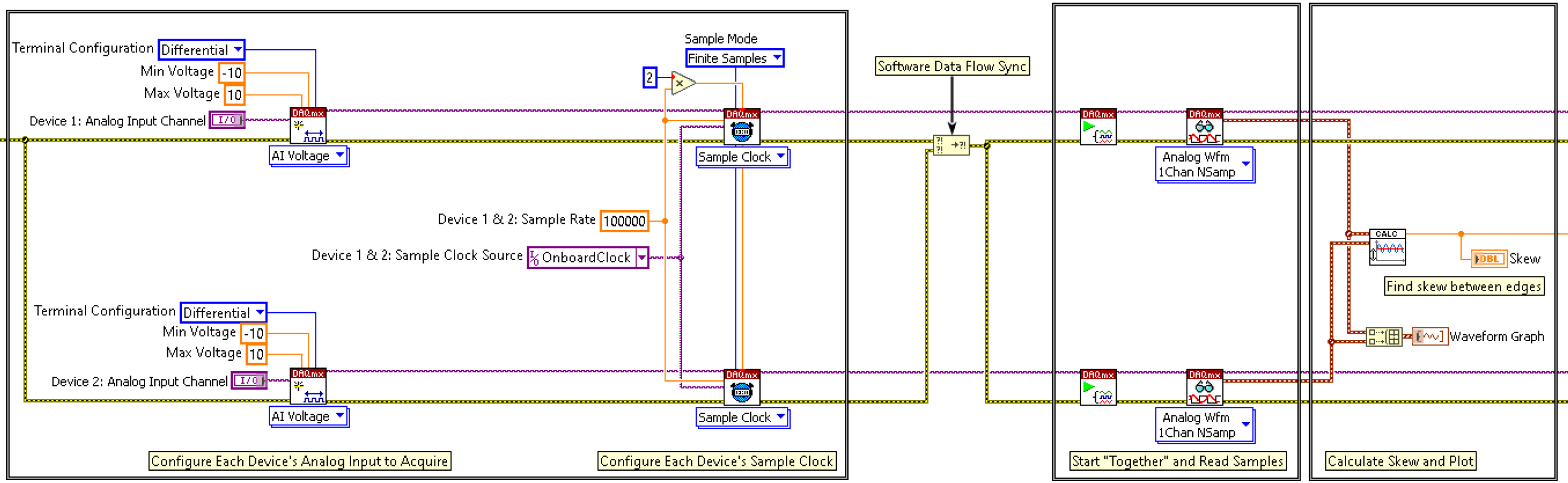
Demo 1: The Wrong Way…

- No Start Triggers
- Local Clocks



Demo 1: The Wrong Way…

- No Start Triggers
- Local Clocks



Demo 1: The Wrong Way…

- No Start Triggers
- Local Clocks

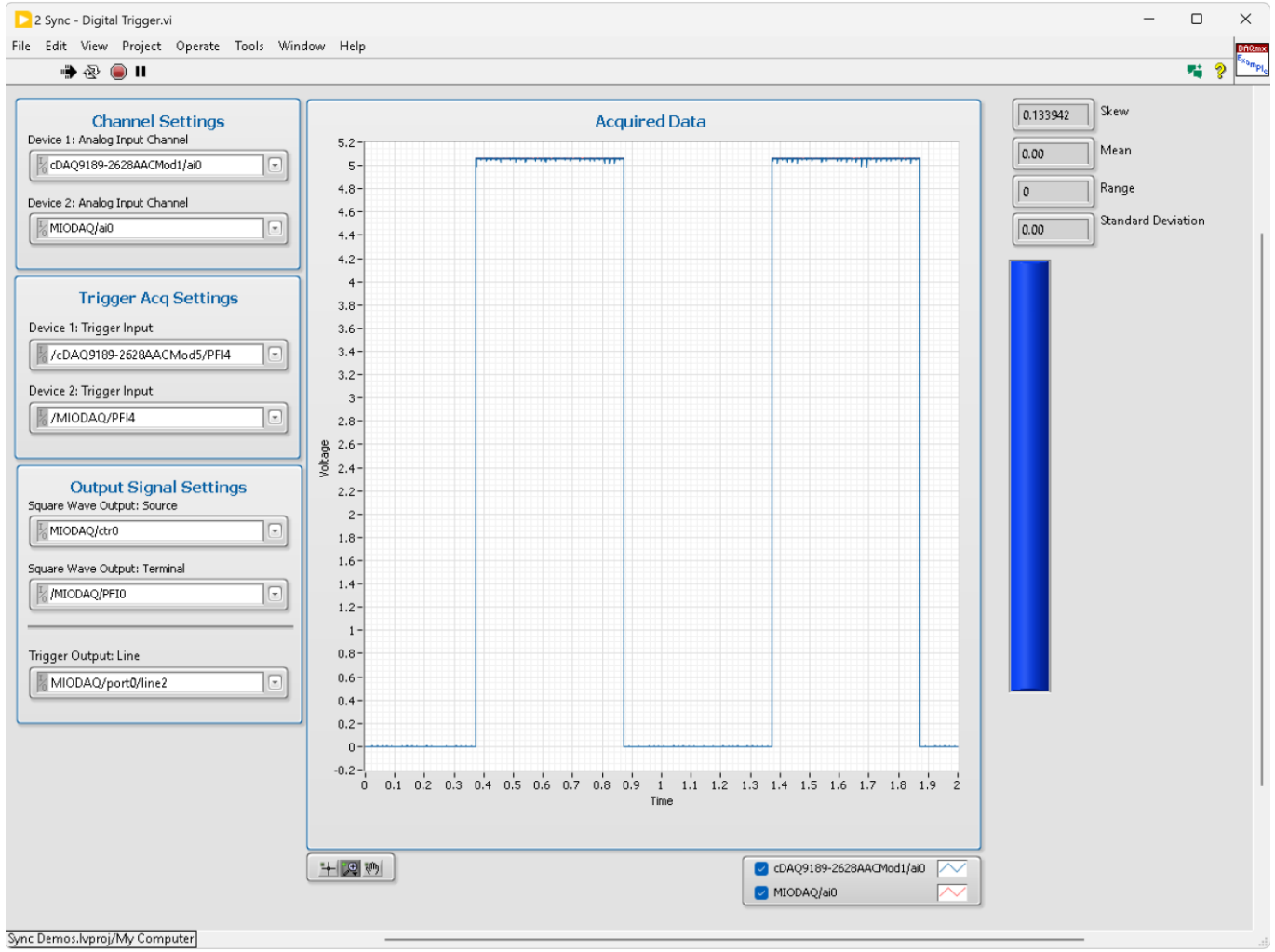
```
# --- Start Tasks "Together" (software sync) ---
ai_task1.start()
ai_task2.start()
```

Approaches to Synchronization

- Share a start trigger → Start devices together
- Share a sample clock → Start devices together; Eliminate drift; Single rate only
- Share a reference clock + Start trigger → Start devices together; Eliminate drift; Multi-rate

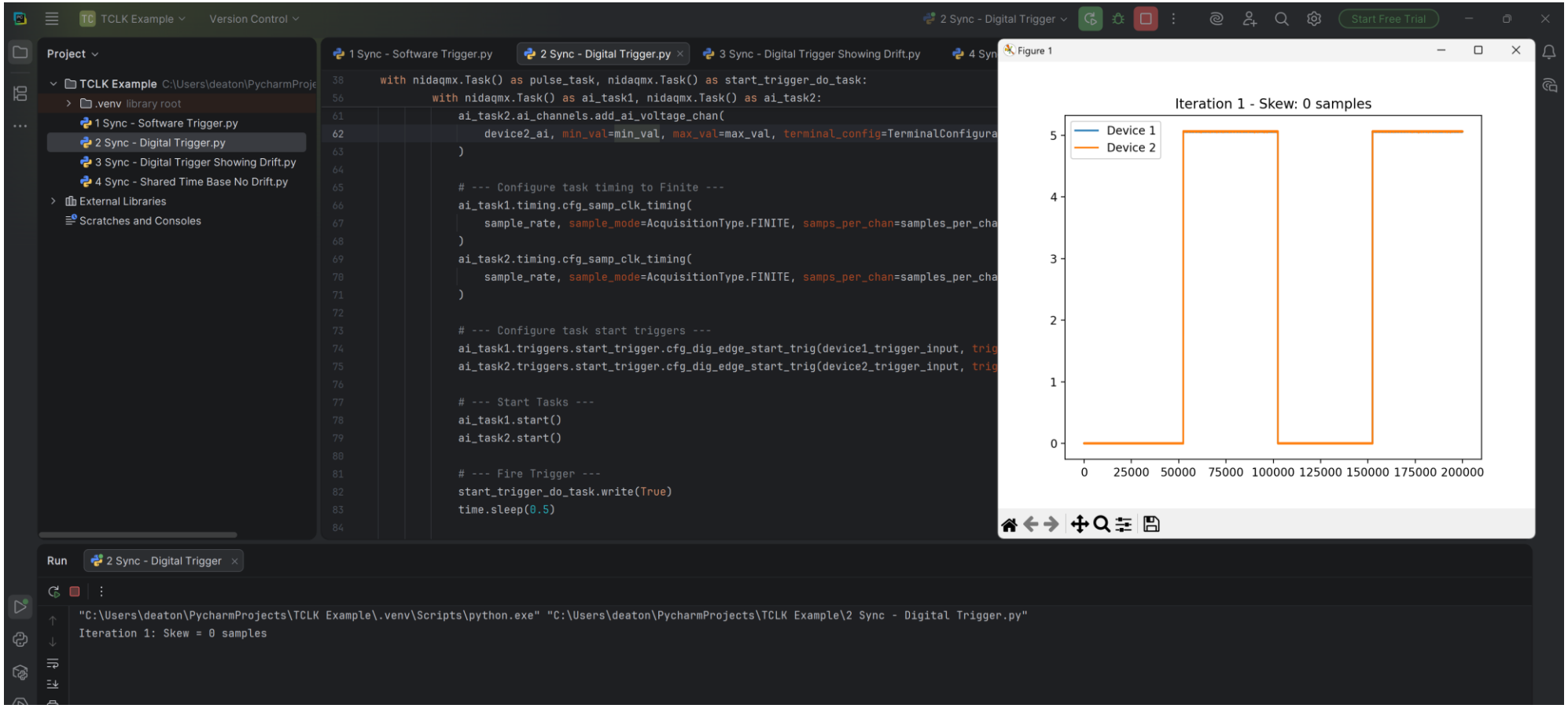
Demo 2: The Wrong Way…But With a Start Trigger

- Share a start trigger → Start devices together!!
- Local Clocks → Drift



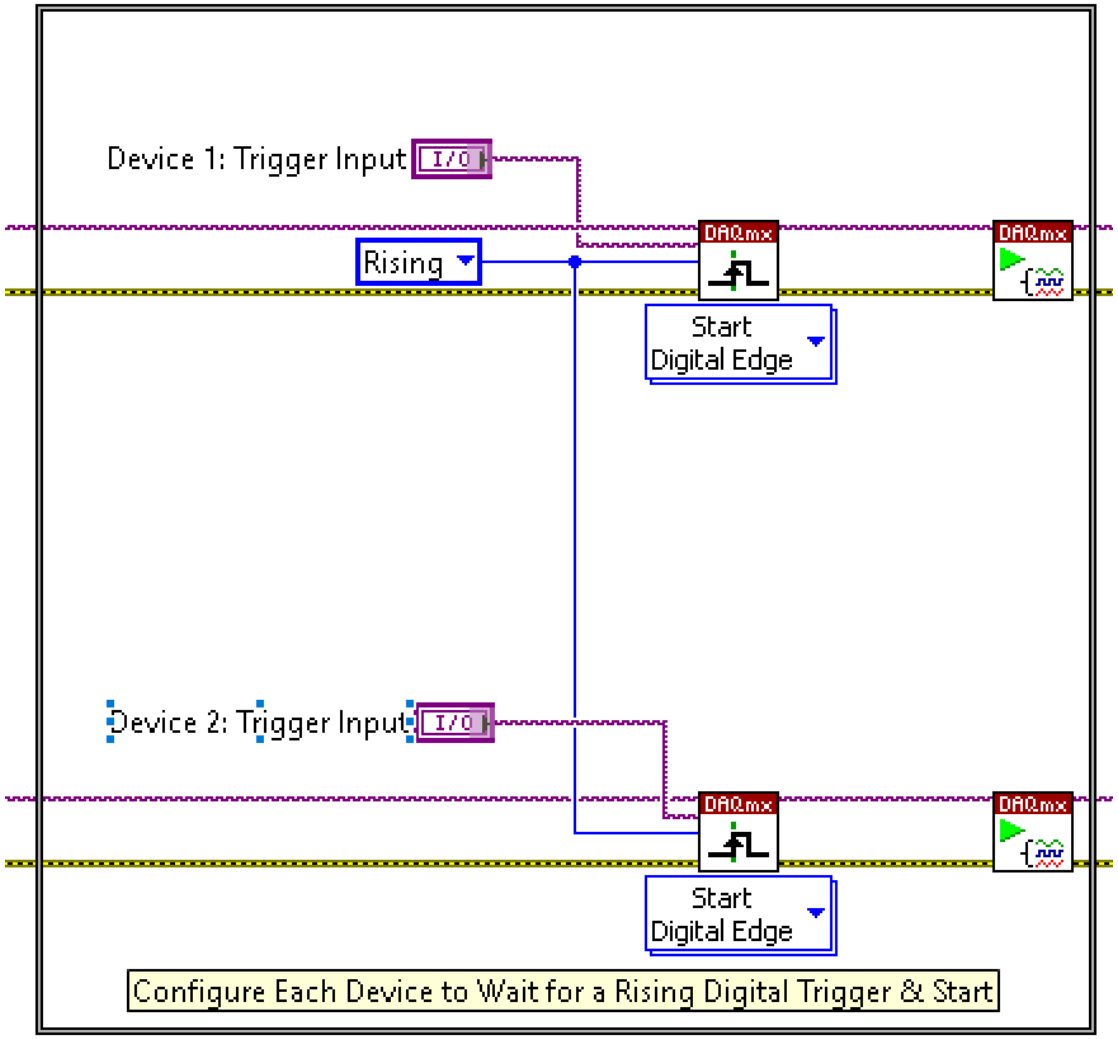
Demo 2: The Wrong Way…But With a Start Trigger

- Share a start trigger → Start devices together!!
- Local Clocks → Drift



Demo 2: The Wrong Way…But With a Start Trigger

- Share a start trigger → Start devices together!!
- Local Clocks → Drift



Demo 2: The Wrong Way...But With a Start Trigger

- Share a start trigger → Start devices together!!
- Local Clocks → Drift

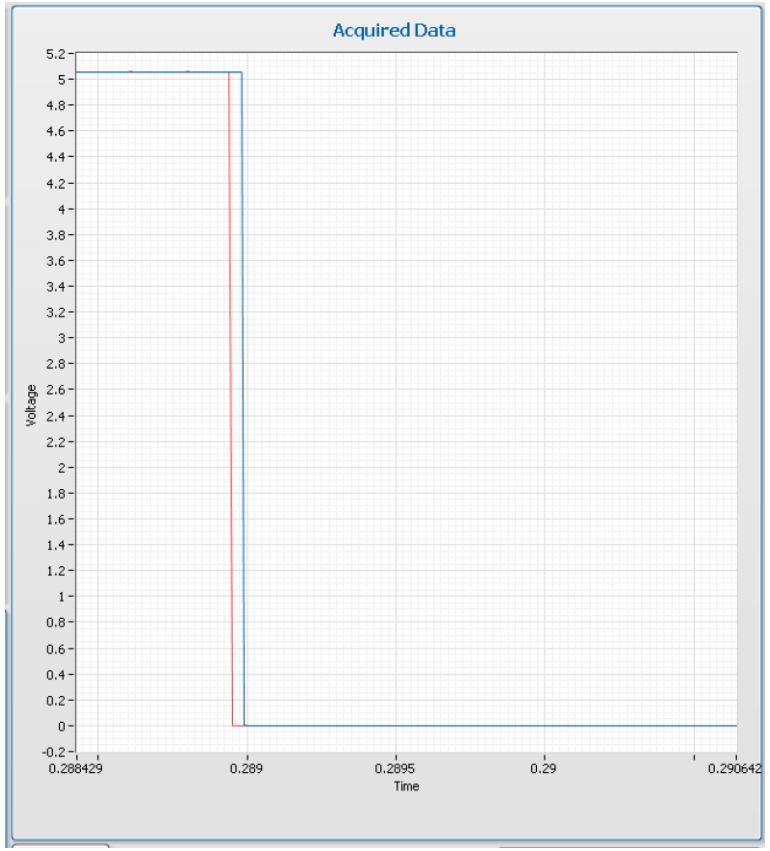
```
# --- Digital Start Trigger User Configuration ---
device1_trigger_output_line = "MIODAQ/port0/line2"
device1_trigger_input = "/MIODAQ/PFI4"
device2_trigger_input = "/cDAQ9189-2628AACMod5/PFI4"
```

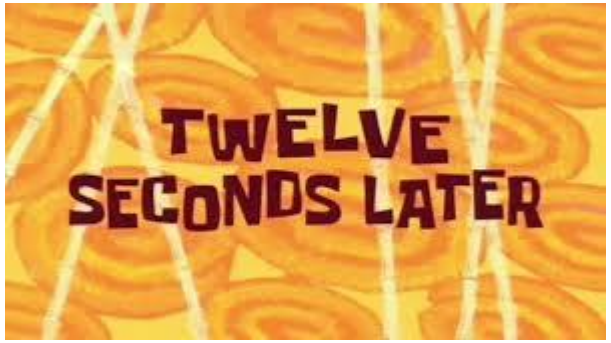
```
# --- Configure Start Trigger Digital Output Task ---
start_trigger_do_task.do_channels.add_do_chan(
    device1_trigger_output_line,
    line_grouping=LineGrouping.CHAN_PER_LINE
)
start_trigger_do_task.write(False)  # Ensure trigger is low
```

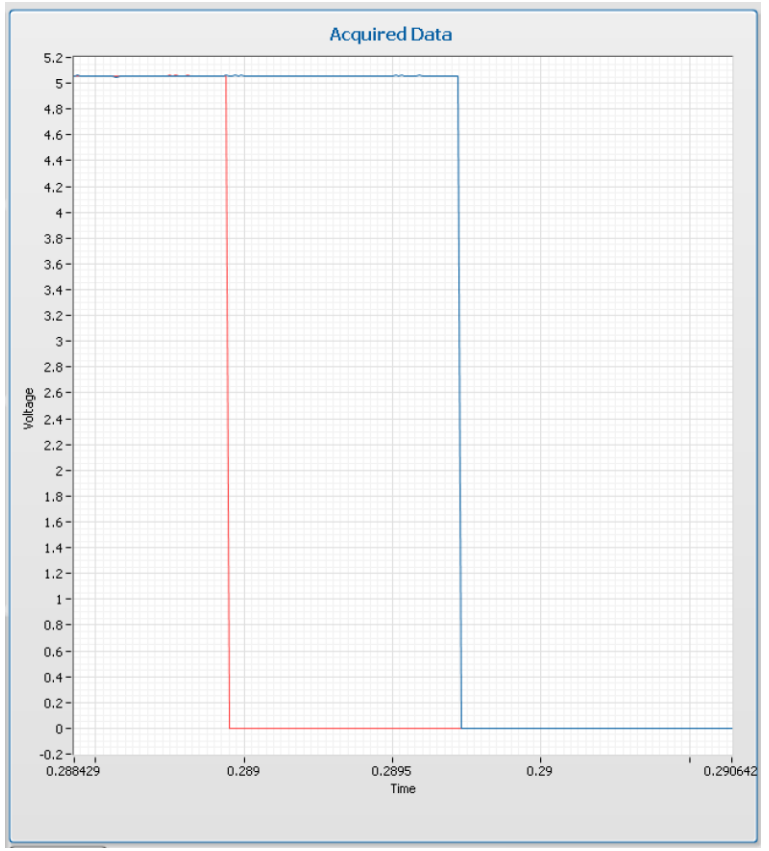
```
# --- Configure task start triggers ---
ai_task1.triggers.start_trigger.cfg_dig_edge_start_trig(
    device1_trigger_input,
    trigger_edge=Edge.RISING
)
ai_task2.triggers.start_trigger.cfg_dig_edge_start_trig(
    device2_trigger_input,
    trigger_edge=Edge.RISING
)

# --- Start Tasks ---
ai_task1.start()
ai_task2.start()

# --- Fire Trigger ---
start_trigger_do_task.write(True)
```

Demo 3: The Wrong Way…But With a Start Trigger

- Share a start trigger → Start devices together!!
- Local Clocks → Drift

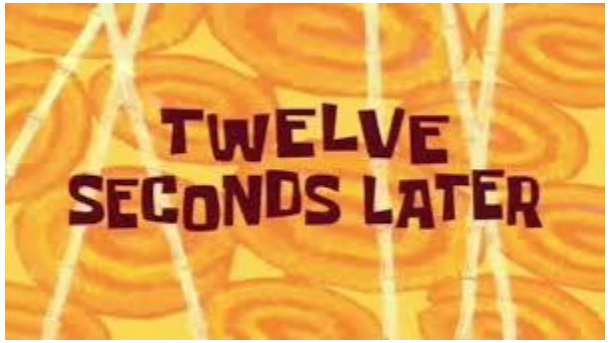






Demo 3: The Wrong Way…But With a Start Trigger

- Share a start trigger → Start devices together!!
- Local Clocks → Drift

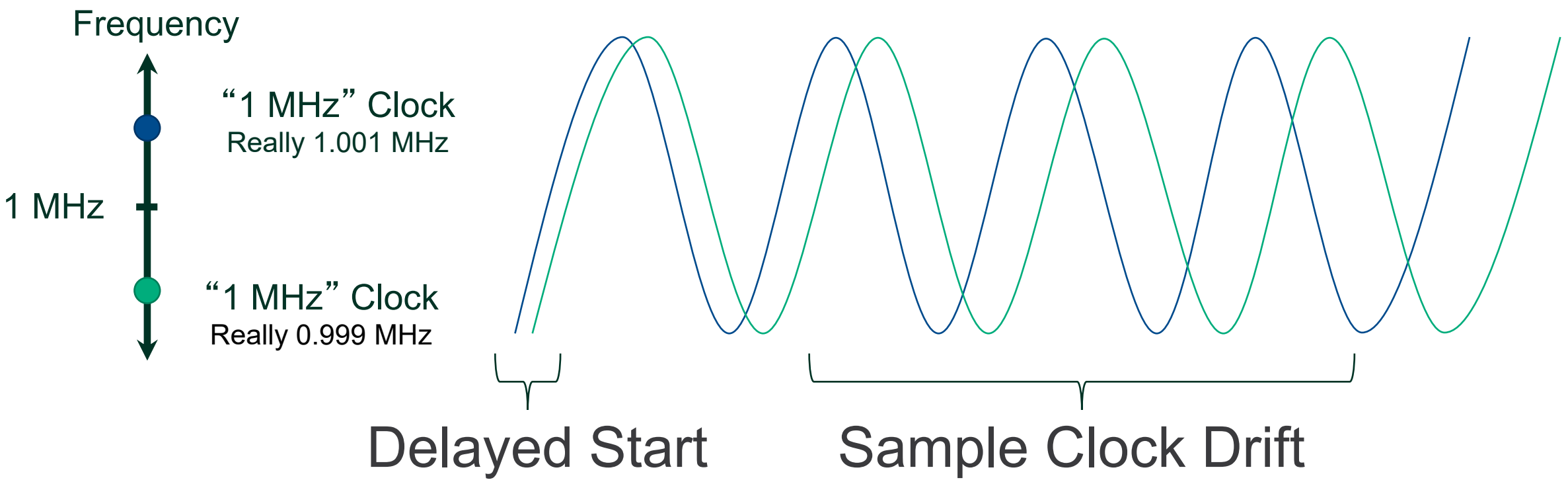


Run 3 Sync - Digital Trigger Showing Drift

```
"C:\Users\deaton\PycharmProjects\TCLK Example\.venv\Scripts\python.exe" "C:\Users\deaton\PycharmProjects\TCLK Example\3 Sync - Digital Trigger Showing Drift.py"
Skew = 0 samples
Skew = -1 samples
Skew = -1 samples
Skew = -1 samples
Skew = -1 samples
Skew = -1 samples
Skew = -2 samples
Skew = -2 samples
Skew = -2 samples
Skew = -2 samples
```

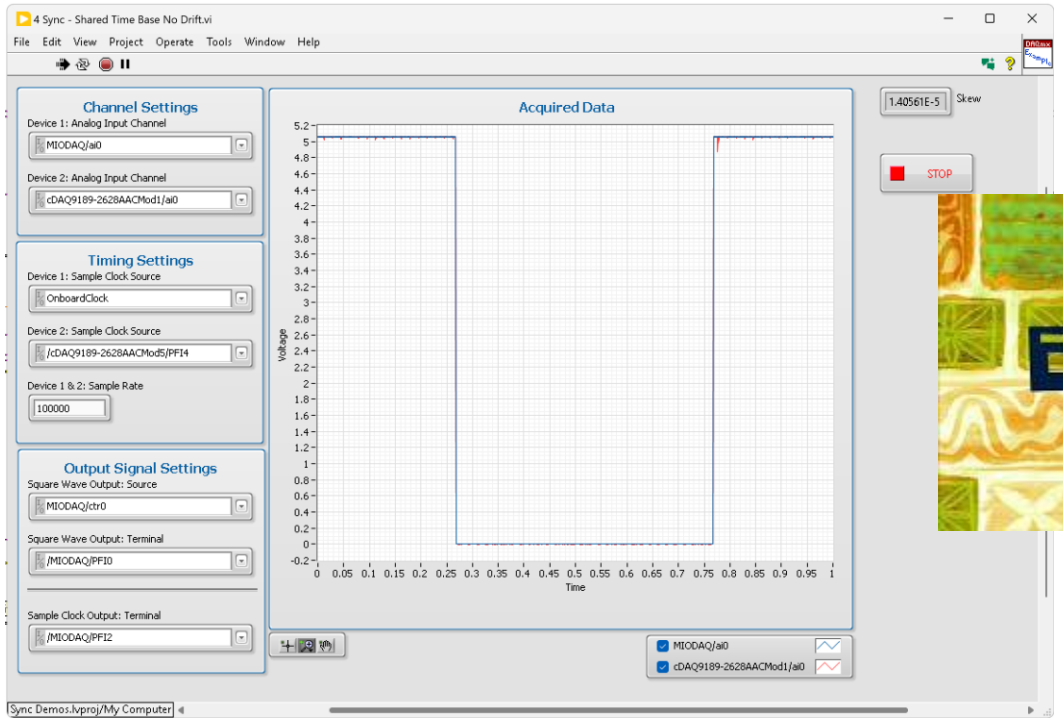
Clock Drift

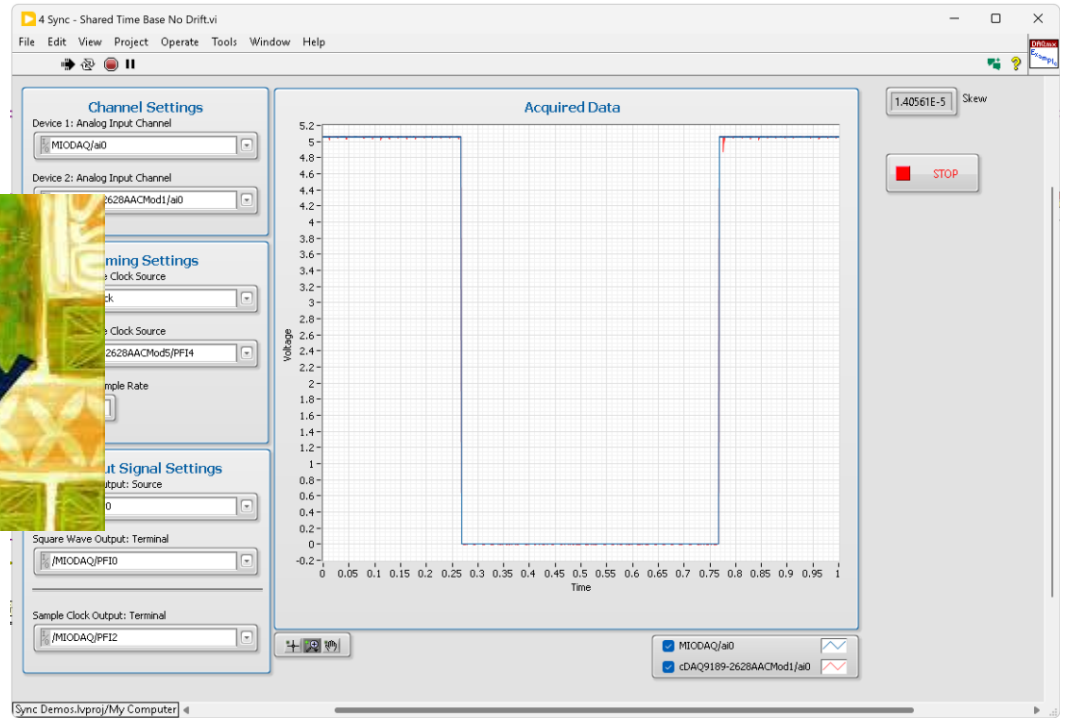
Errors grow as acquisition continues.



Demo 4: Almost…There…But SKEW

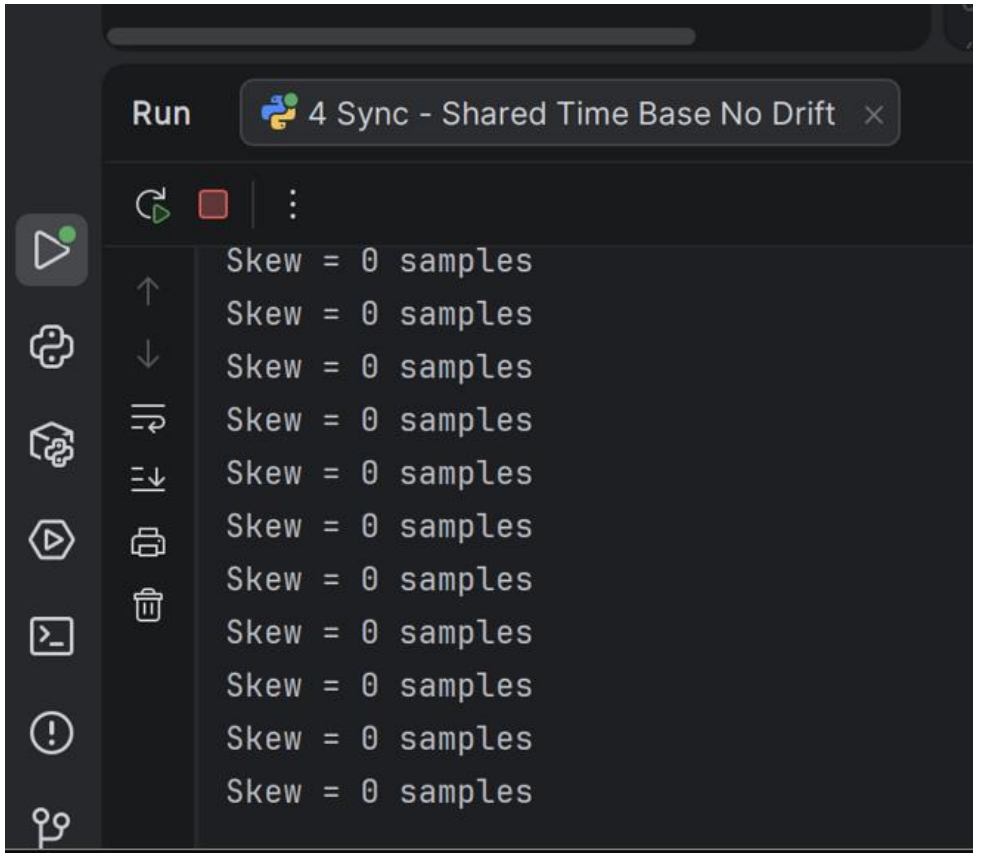
- Shared Clock → Start devices together; Eliminate drift; Single rate only; Still SKEW





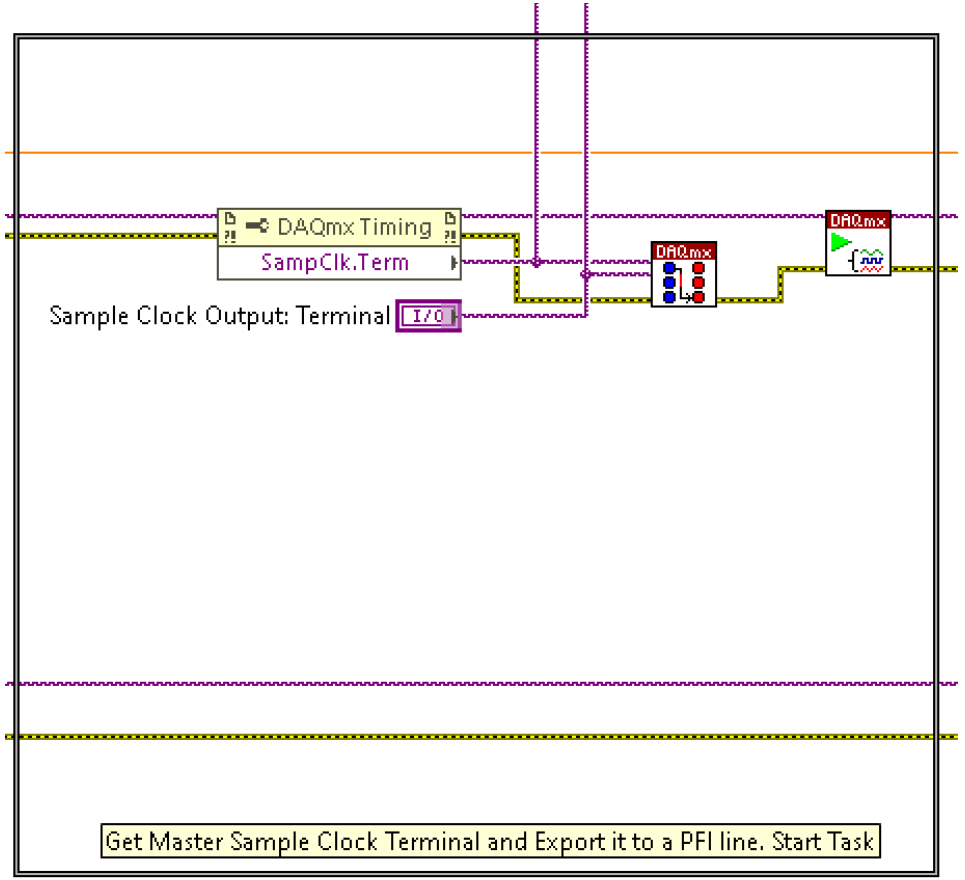
Demo 4: Almost…There…But SKEW

- Shared Clock → Start devices together; Eliminate drift; Single rate only; Still SKEW



Demo 4: Almost…There…But SKEW

- Shared Clock → Start devices together; Eliminate drift
- Single rate only
- SKEW!!



Signal Path for Clock

Device 1: Internal Clock

Device 2: Device 1 Internals → PFI line → Wire → 9401 Acq → Digital Level Logic → cDAQ Backplane → 9205

Demo 4: Almost…There…But SKEW

- Shared Clock → Start devices together; Eliminate drift
- Single rate only
- SKEW!!

```
# --- Analog Input Sample Clock User Configuration ---
device1_sample_clock = ""
device1_exported_sample_clock_term = "/MIODAQ/PFI2"
device2_sample_clock = "/cDAQ9189-2628AACMod5/PFI4"
```

```
# --- Configure task timing to Continuous & Use Set Clock Source ---
ai_task1.timing.cfg_samp_clk_timing(
    sample_rate,
    source=device1_sample_clock,
    sample_mode=AcquisitionType.CONTINUOUS,
    samps_per_chan=10*samples_per_channel
)
ai_task2.timing.cfg_samp_clk_timing(
    sample_rate,
    source=device2_sample_clock,
    sample_mode=AcquisitionType.CONTINUOUS,
    samps_per_chan=10*samples_per_channel
)

# --- Start Task2 ---
ai_task2.start()

# --- Export Task1 clock to an output terminal; externally wired to device 2---
ai_task1.export_signals.samp_clk_output_term = device1_exported_sample_clock_term
ai_task1.start()
```

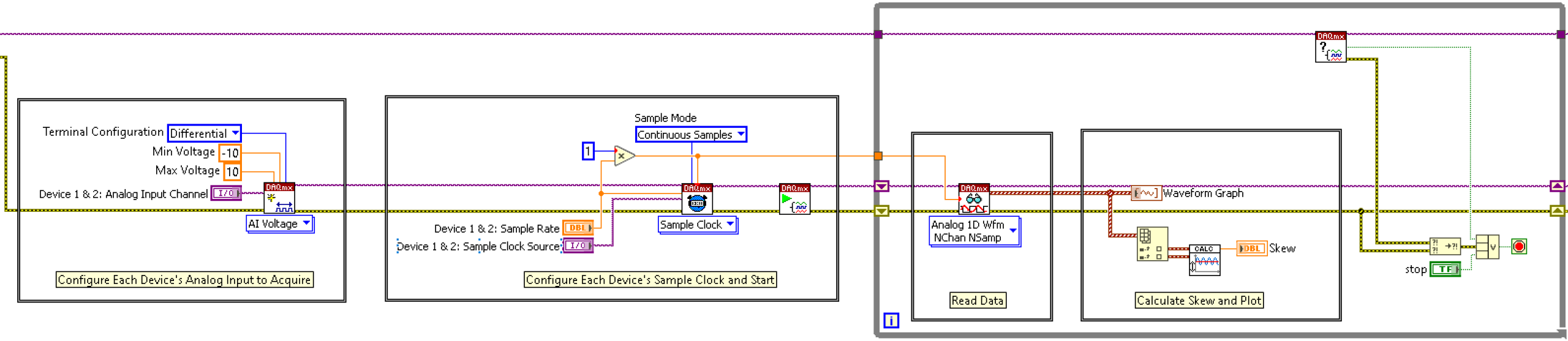
Signal Path for Clock

Device 1: Internal Clock

Device 2: Device 1 Internals → PFI line → Wire → 9401 Acq → Digital Level Logic → cDAQ Backplane → 9205

Demo 5: CALLBACK to Know Requirements → Pick Approach

- NI TSN Easy Button…For General DAQ Applications
- Agree on time → Start Together and Stay Together…Simpler Code



The most-right way

- Share a reference clock + Start trigger → Start devices together; Eliminate drift; Multi-rate
  - Triggers must have length matched cables
  - Highly Stable, Master Reference Clock with length matched cables
  - Routed to every device (no internal clocks allowed…)
  - Each device divides down or syncs to Master Reference Clock (multi-rate!!)
- Might be good enough
  - …You will still have skew…just much less…
  - …You will still have edge detection inconsistencies…

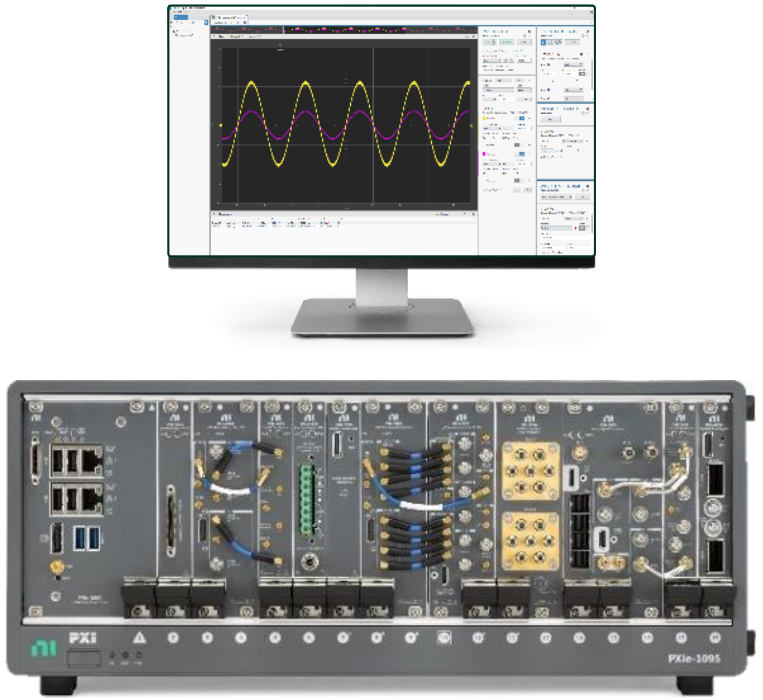
DOUBLE CALLBACK COMING…

Different Approaches

Common Methods – Different Approaches

Boxed Instruments

Supported: Device Dependent
Wiring: External, Manual
Config: Different per device type



PXI Modular Instruments

Supported: Yes
Wiring; Internal, Software Defined
Config: Common API

Integrated Timing & Synchronization With PXI

- Requires length matched cables for clocks and triggers
- Static configuration; no flexibility
- Extremely difficult with multiple instrument types or clock rates

- Everything is built into the backplane; no external cabling
- NI driver SW manages all resources with pico-second level accuracy
- Multiple instrument types and clock rates are trivial to combine

The PXI Approach

PXI Hardware Tools for Synchronization

The PXI backplane

- The PXI chassis contains resources to enable sophisticated synchronization, timing, and triggering.

NI PXI timing and synchronization modules

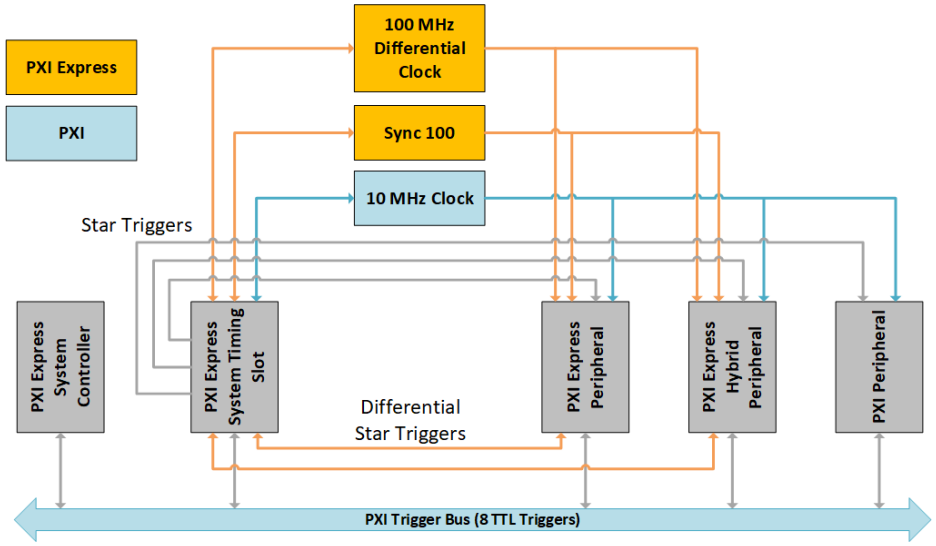
- These are specialized modules which interface to PXI backplane timing and synchronization resources for advanced coordination and control of measurement and signal generation timing.

The PXI Backplane

Available resources:

- Clocks
- Triggers
- Timing Slots

Some resources may only be available to PXI Express or PXI Express Hybrid slots



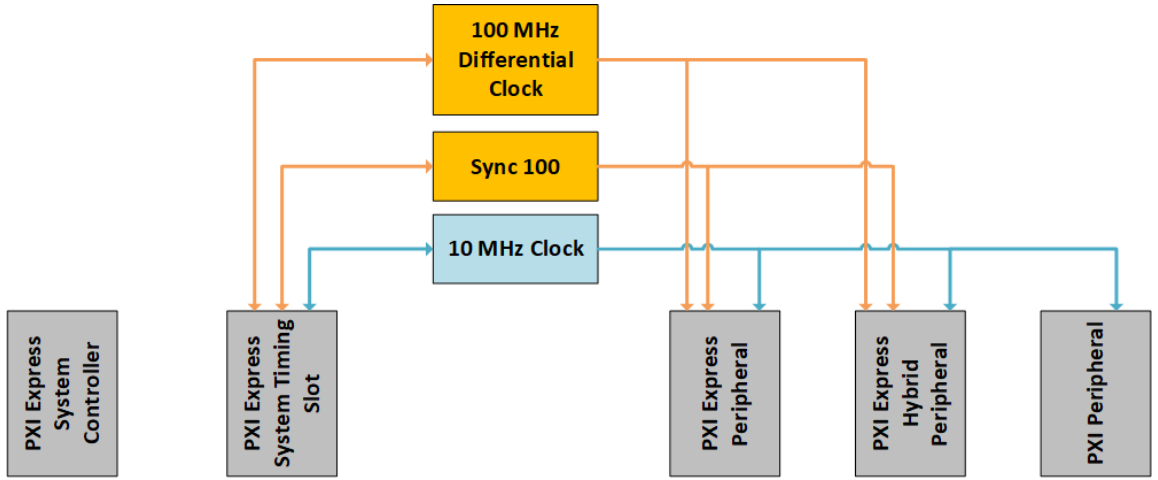
PXI and PXI Express Backplane Synchronization Resources

	PXI	PXI Express
Backplane Clock	10 MHz	10 MHz, 100 MHz differential
Trigger Bus	8x single-ended trigger bus*	8x single-ended trigger bus*
Star Triggers	1x single-ended star trigger	1x singled ended star trigger, 8x differential star triggers

PXI Backplane Clocks

Provide common reference clocks to all module slots

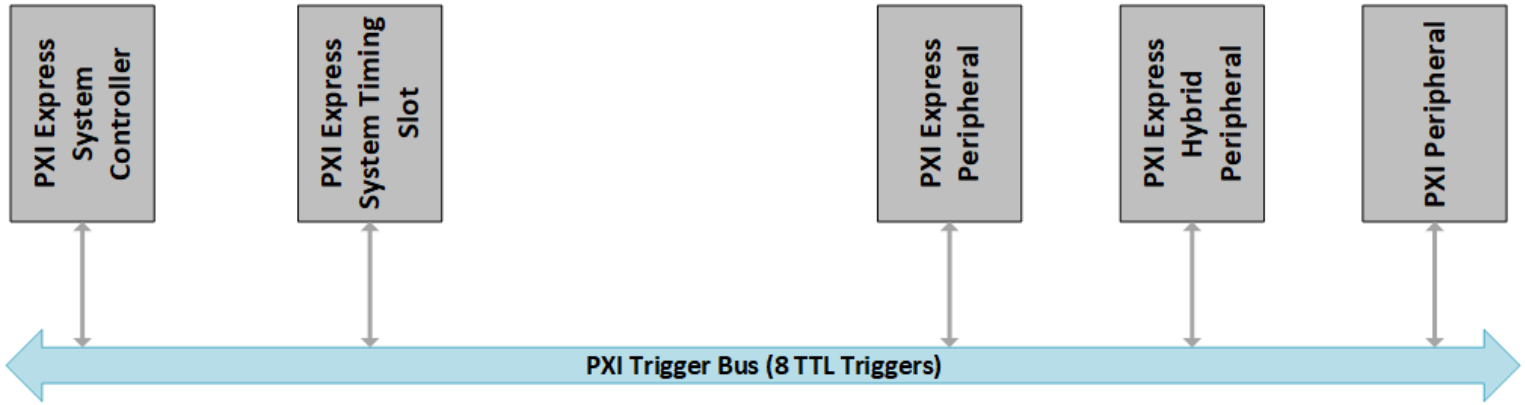
- 10 MHz clock provides <1 ns device-to-device skew
- 100 MHz differential clock provides <100 ps device-to-device skew
- 100 MHz differential clock provides higher frequency ceiling and increased noise immunity over 10 MHz clock



PXI Backplane Single-Ended Trigger Bus

General purpose synchronization lines accessible to all module slots

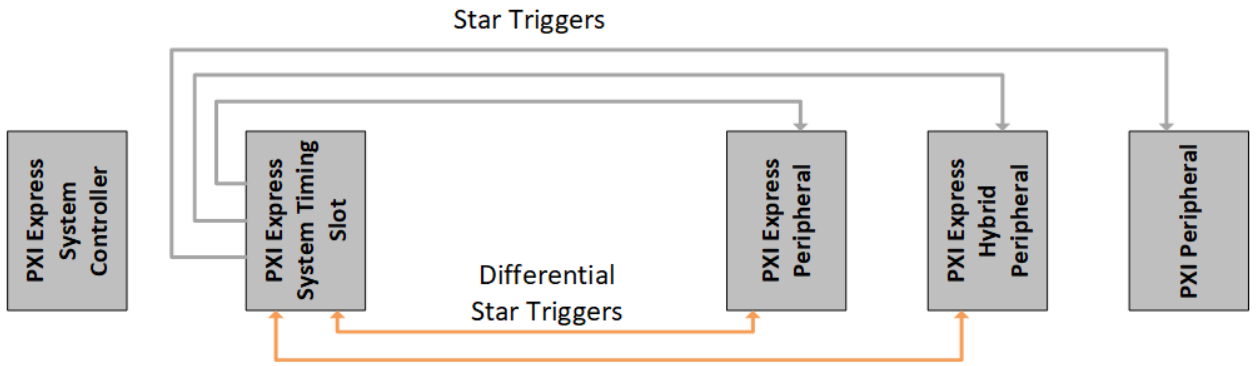
- Potentially significant propagation delay depending on signal loading and module position
- Signal integrity may vary depending on hardware and configuration
- Best used for trigger signals where device-to-device skew of up to 80 ns is acceptable



PXI Backplane Star Triggers

High-performance triggers available to all module slots

- Can only be controlled from specific slots
- Traces are length-matched and allow propagation delays of <5 ns and device-to-device skew of <1 ns
- Best used for applications which require precise triggering to multiple modules

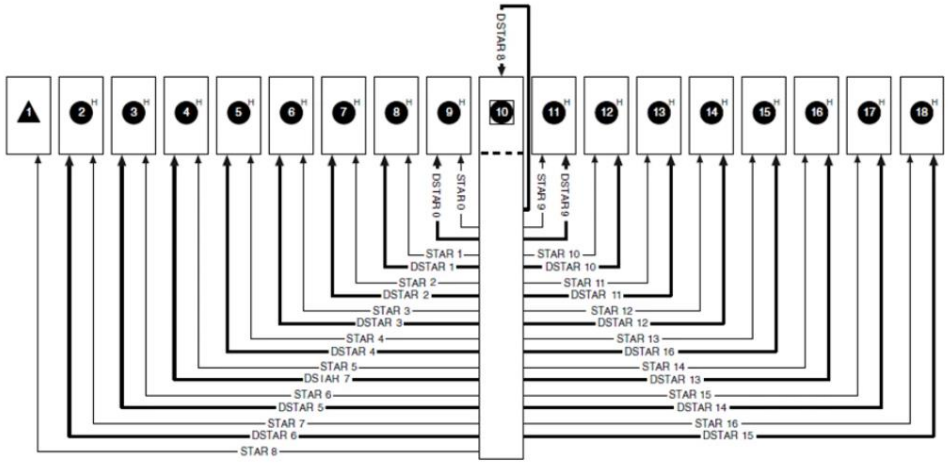


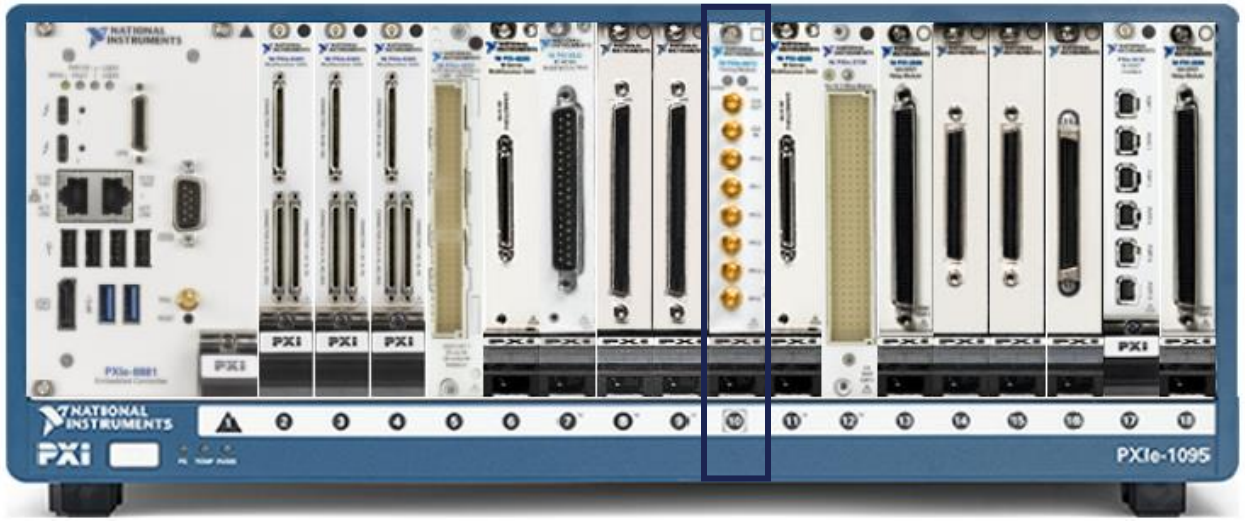
PXI Chassis Timing Slots

Offer direct access to star triggers, trigger bus, and backplane clocks for timing and synchronization PXI modules

Timing and synchronization modules installed in chassis timing slots may import external clock signals and override the 10 MHz backplane clock

Star triggers **require** a timing and synchronization module installed in a chassis timing slot





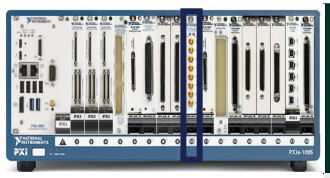
NI Timing and Synchronization Modules

- Import and export reference clock signals directly to/from the PXI backplane
- Generate high quality clock signals
- Access to PXI trigger lines and star triggers
- Synchronize to an external time reference
- Requirements Change…Swap the card

Need Time Based Sync?
NI PXI 6683H

Need Signal Based Sync?
NI PXIe 6674

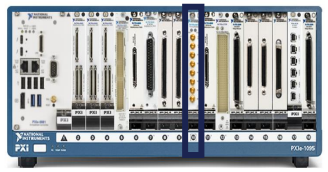
Multi-Chassis Synchronization



PXI-Chassis (Master)

Length-Matched Cabling

PXI-Chassis (Slave)



PXI-6674 (Sync)

ClkOut

ClkIn

PFI0

PFI1

ClkIn

PFI0

PXI-6674 (Sync)

EMERSON

Approaches Compared

Propagation Delay

(Assuming devices are capable)

- Length Matched, External Wiring
- Does not account for internal device delay

- External Triggers/Clocks wire to Sync Module
- Sync Module route to PXI backplane
  - Length Matched, Internal Wiring
- Highest speed devices provide internal skew compensation

EMERSON

Drift

Occurs when two instruments are acquiring data at different sample rates

Error builds over time

Drift

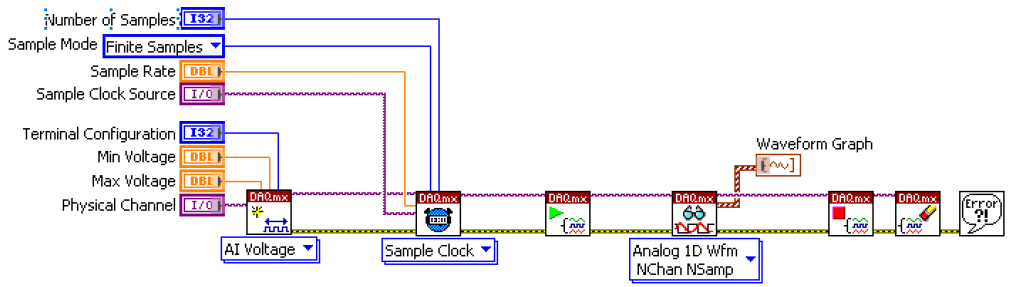
(Assuming devices are capable)

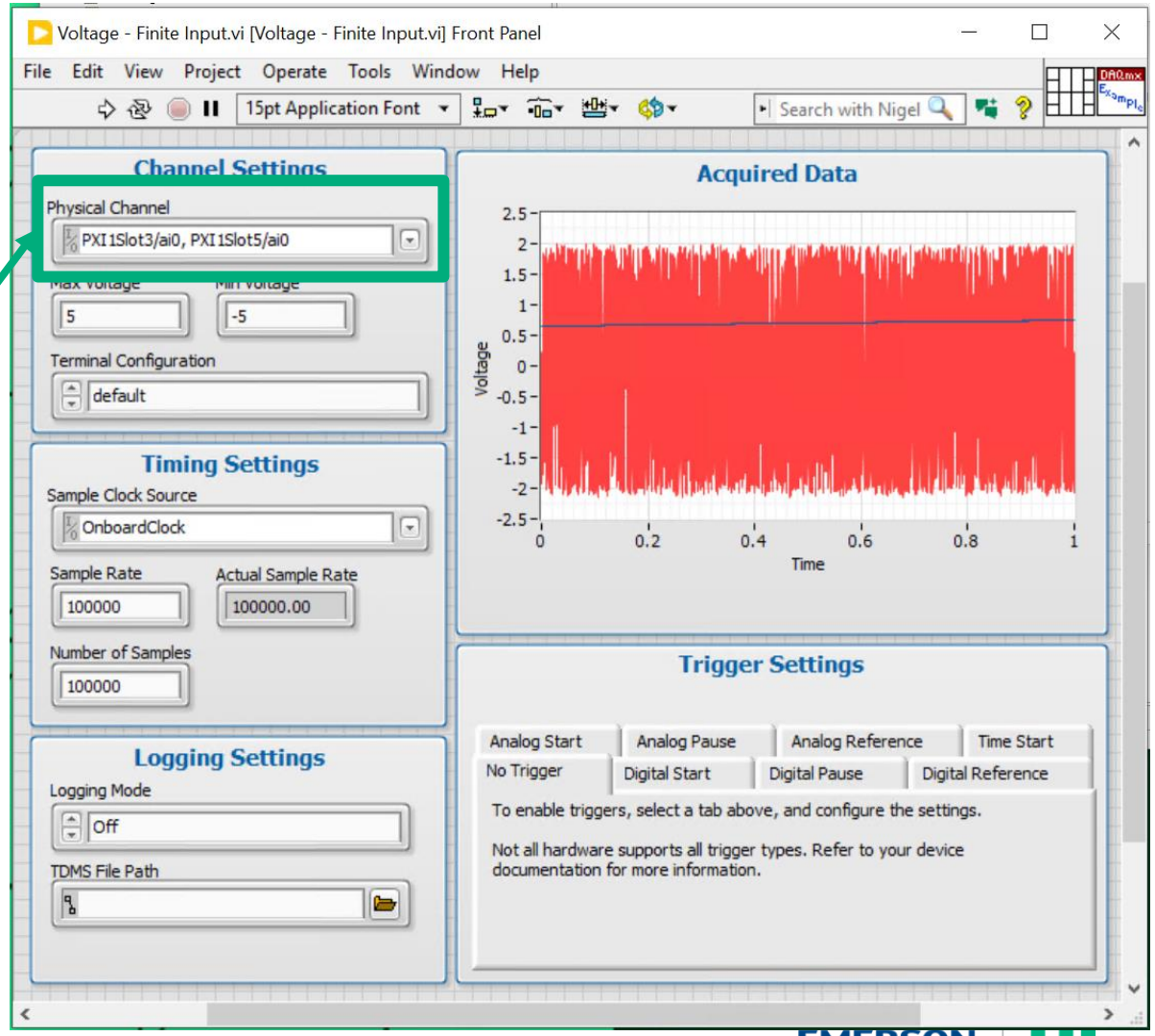
- Buy / export a master clock
- Import the clock on other devices
- Length Matched, External Wiring
- Configure each device’s software

- PXI Chassis includes a 10 MHz clock
- Devices automatically sync (PLL)
- (Or) Share dedicated clock over internal lines
- (Either way) Can’t drift even if you wanted to

CALLBACK #1…Automatic Synchronization with NI DAQmx

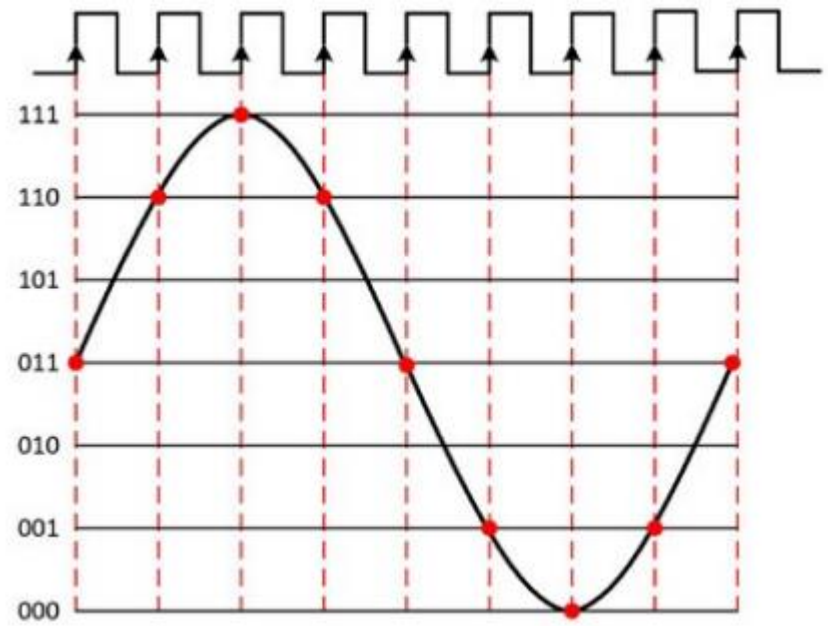
- PXI Chassis contains 10 MHz Clock
- Each DAQmx card PLLs to 10 MHz
- Internal Start Triggers Automatically Routed
- Result:
  - Multiple Devices in a Single Task (like TSN)
  - Start Together
  - Stay Together

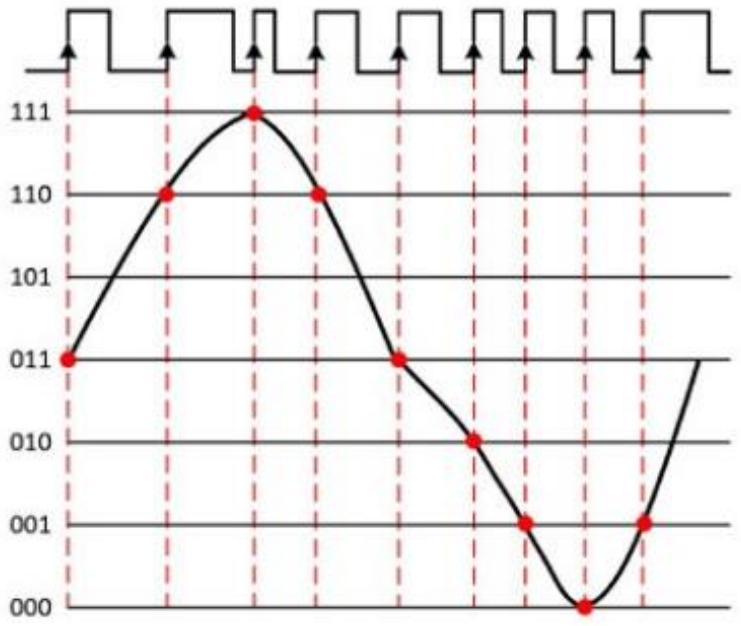




Jitter

Jitter is the deviation from the ideal timing of an event to the actual timing of an event.





Jitter

- Buy a better clock

- Buy a better clock
  - Option 1: PXI Chassis (Link)

  - Option 2: Timing / Sync Module
    - PXI-6674T (Link)

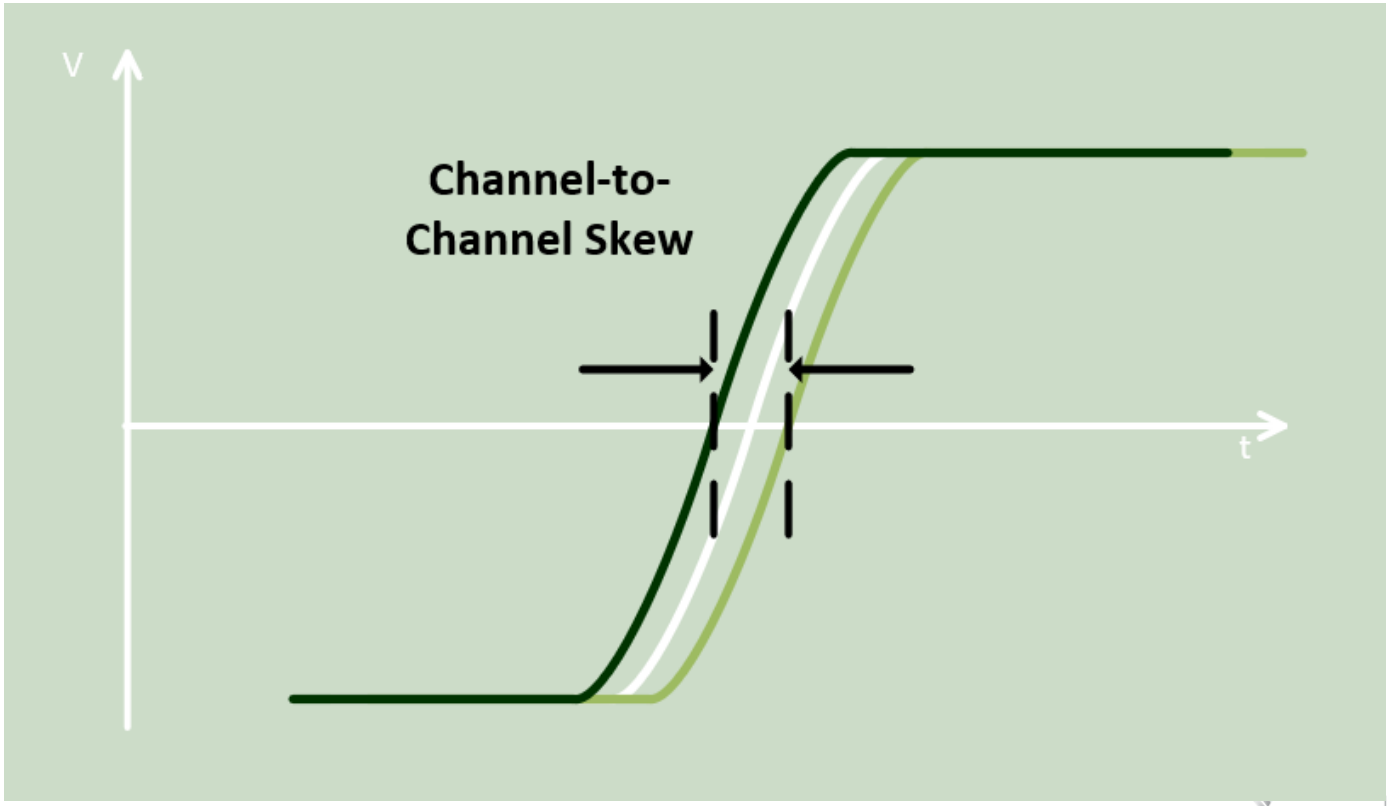
Skew

Skew is when the clock signal arrives at different components at different times.

Maintains clock period

Causes:

- Wire Length
- Temperature Variation
- Different Input Capacitance



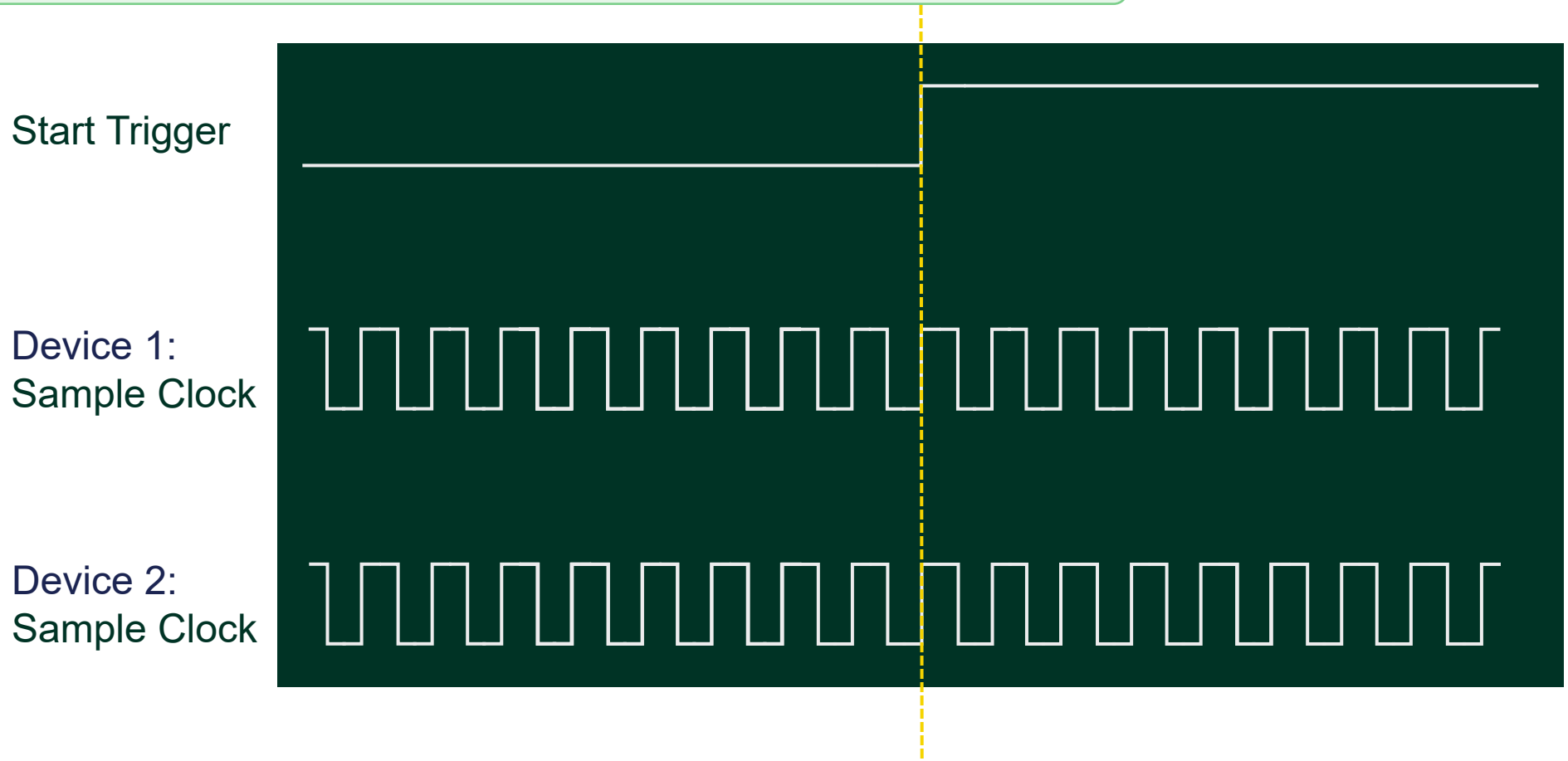
Skew

- Manually calibrate system
- Manually calculate skew
- Buy a digital delay generator
- Delay generation of clocks and trigger
- (Or) Adjust in device; per device type

- NI TCLK – Automatically handles it

Edge Detection Inconsistencies

Distributing start time triggers and handling metastability



Edge Detection Inconsistencies

- ??

- NI TCLK – Automatically handles it

Configuration of all the Things

- External Wiring
- Unique in-device configurations
- Unique SCPI APIs per device family

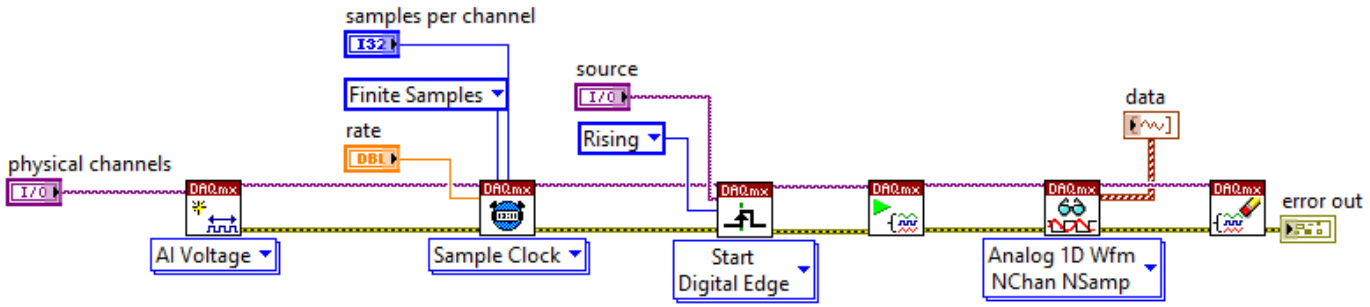
- Base: Device HAL API (e.g. NI DAQmx)
  - All NI Drivers are Hardware Abstraction Layers
  - USB, cDAQ/cRIO, PXI, FieldDAQ → NI DAQmx
- External Signals: Device HAL API + NI Sync API
- Patented Black Magic: NI TCLK

NI APIs Offering Synchronization Capabilities

NI-DAQmx	NI-Sync	NI-TClk
Out-of-the-box synchronization for DAQ-based applications	Control and configuration of NI timing and synchronization modules	Synchronization for high-speed and trigger-critical applications

NI-DAQmx Capabilities

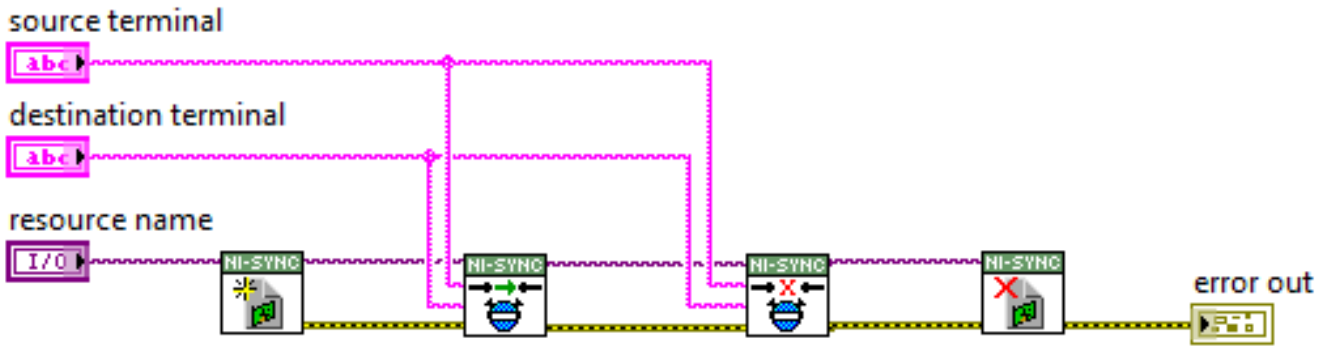
- Provides general-purpose synchronization for DAQ applications
- Supports most analog, digital, and multifunction IO PXI modules
- Supports mixed-mode synchronization
- Supports onboard timing sources and timing sources applied from external sources
- Enables synchronization between signals on the same device and signals hosted on multiple devices
- Supports analog and digital triggering
- Provides automatic synchronization signal routing



EMERSON | NI

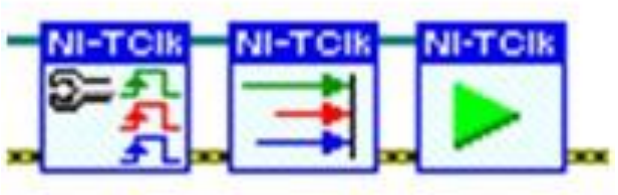
NI-Sync Capabilities

- Configure NI timing and synchronization modules and use them in conjunction with other devices
- Route clocks and triggers between devices or chassis
- Generate reference clocks and triggers
- Synchronize to an external time reference (e.g., IEEE-1588, IRIG, GPS)



NI-TClk Capabilities

- Synchronization for instruments which perform high-speed measurements
- Ensure that devices respond to triggers at the same time
- Synchronize across multiple devices and chassis
- Synchronize to an internal or external sample clock
- Synchronize across devices which have differing sample clock rates



Text Based Options: NI APIs Offering Synchronization Capabilities

(Windows Support)	Python (Link)	C/C++ & .NET (Link)	gRPC Support (Link)
NI DAQmx	Link	Included in Driver	Yes
NI Sync	Link	Included in Driver (C/C++ only)	Yes
NI TCLK	Link	Included in Driver	Yes
NI SCOPE	Link	Included in Driver	Yes
NI FGEN	Link	Included in Driver	Yes

NI TCLK...CALLBACK #2

Picosecond Synchronization

EMERSON

Pesky Problems Remain...For Everyone...

- NI TCLK To the Rescue!

Synchronization Challenges

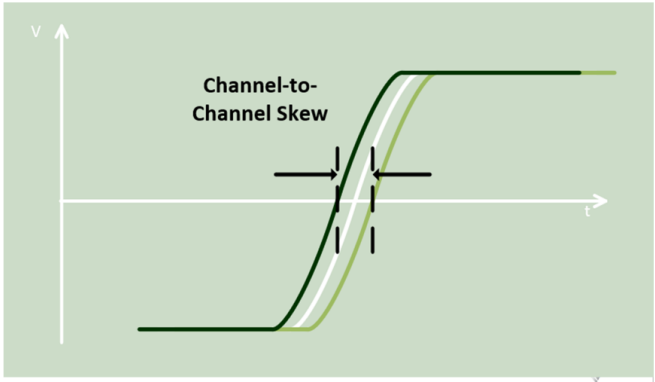
Skew

Skew is when the clock signal arrives at different components at different times.

Maintains clock period

Causes:

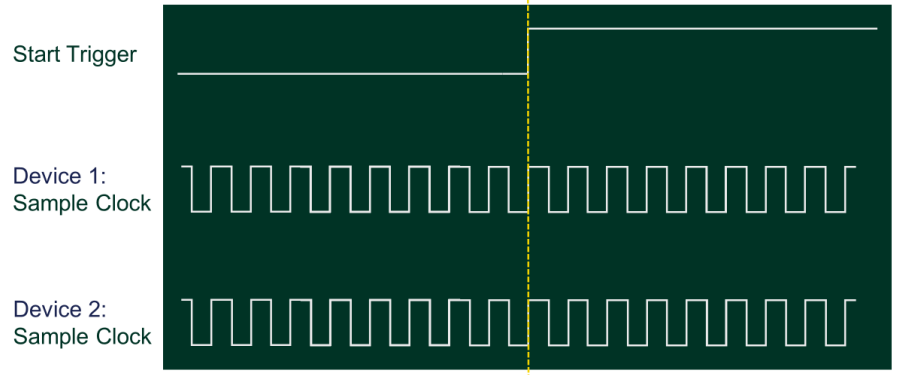
- Wire Length
- Temperature Variation
- Different Input Capacitance



Synchronization Challenges

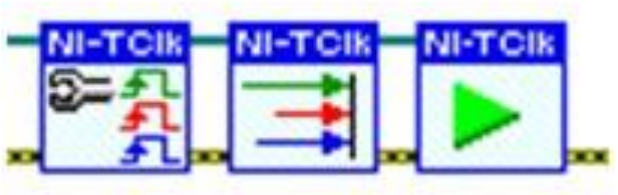
Edge Detection Inconsistencies

Distributing start time triggers and handling metastability



NI-TClk Capabilities

- Synchronization for instruments which perform high-speed measurements
- Ensure that devices respond to triggers at the same time
- Synchronize across multiple devices and chassis
- Synchronize to an internal or external sample clock
- Synchronize across devices which have differing sample clock rates



NI-TClk

- Software library targeted at high-speed signal generation and acquisition instruments
- Allows devices to respond to triggers nearly simultaneously
- Does not require devices to share clocks
- Supports synchronization among devices on single or multiple chassis

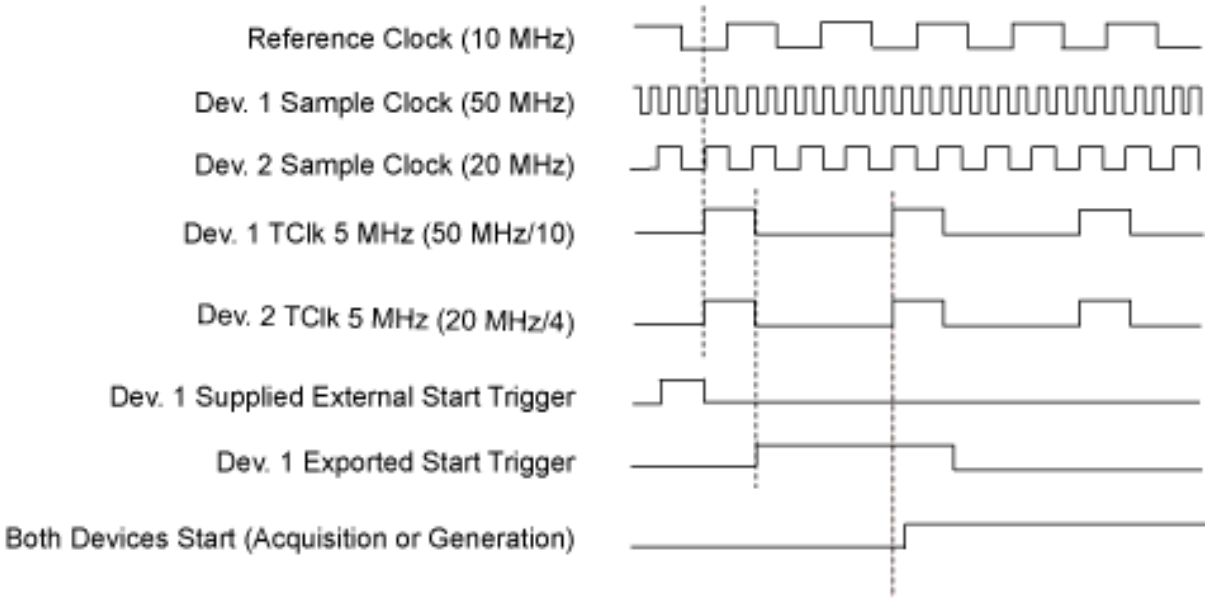
NI-TClk Operation

1. Devices start with initially unaligned sample clocks.
2. Each device generates its own TClk signal whose period is determined by the software based on the sample clock periods of all devices included in the TClk group.
3. Device sample clocks are phase-locked to the PXI 10 MHz reference clock which serves as the Sync Pulse Clock (or, optionally, to a high-precision externally supplied clock).
4. A Sync Pulse is generated, and each device waits for the first rising edge of the Sync Pulse Clock after its receipt.
5. The time between this rising edge and the first rising edge of the device's local TClk signal is measured, and the offsets are used to adjust device sample clocks and TClks to bring them into alignment.

Sample Clock, TClk, and Trigger Relation

Trigger is distributed on the falling edge of TClk signal.

Devices begin acquisition/generation on next TClk rising edge.

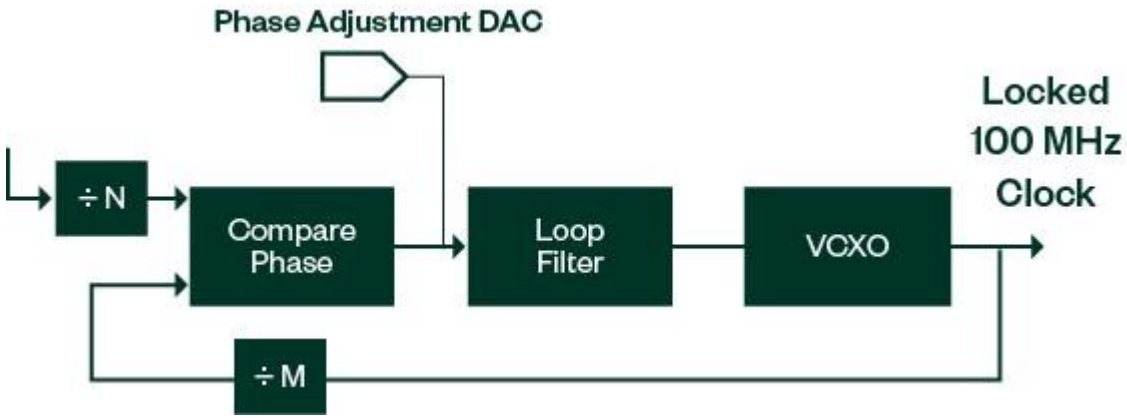


Device Correction

TCLK Driver automatically updates each device's internal phase adjustment

User can calibrate and tune these values for 10s of picosecond synchronization

Make sure your device supports TCLK (if your application requires it)



Supported PXI Cards

- Link

Device Type	Model Number
Arbitrary Waveform and Function Generators	5402, 5406, 5412, 5413, 5421, 5422, 5423, 5433, 5441, 5442, 5450, 5451
High-Speed Digitizers	5105, 5108, 5110, 5111, 5113, 5114, 5122, 5152, 5153, 5154, 5160, 5162, 5163, 5164, 5170, 5171, 5172, 5185, 5186, 5622, 5624, 5763, 5764, 5774, 5775, 5922
High-Speed Digital I/O	6541, 6542, 6544, 6545, 6547, 6548, 6551, 6552, 6555, 6556, 6561, 6562
Digital Pattern Instruments	6570, 6571
FlexRIO	5745, 5763, 5764, 5774, 5775, 5785
RF Devices and Modules	5665, 5668, 5741, 5820, 5830, 5831, 5840, 5842

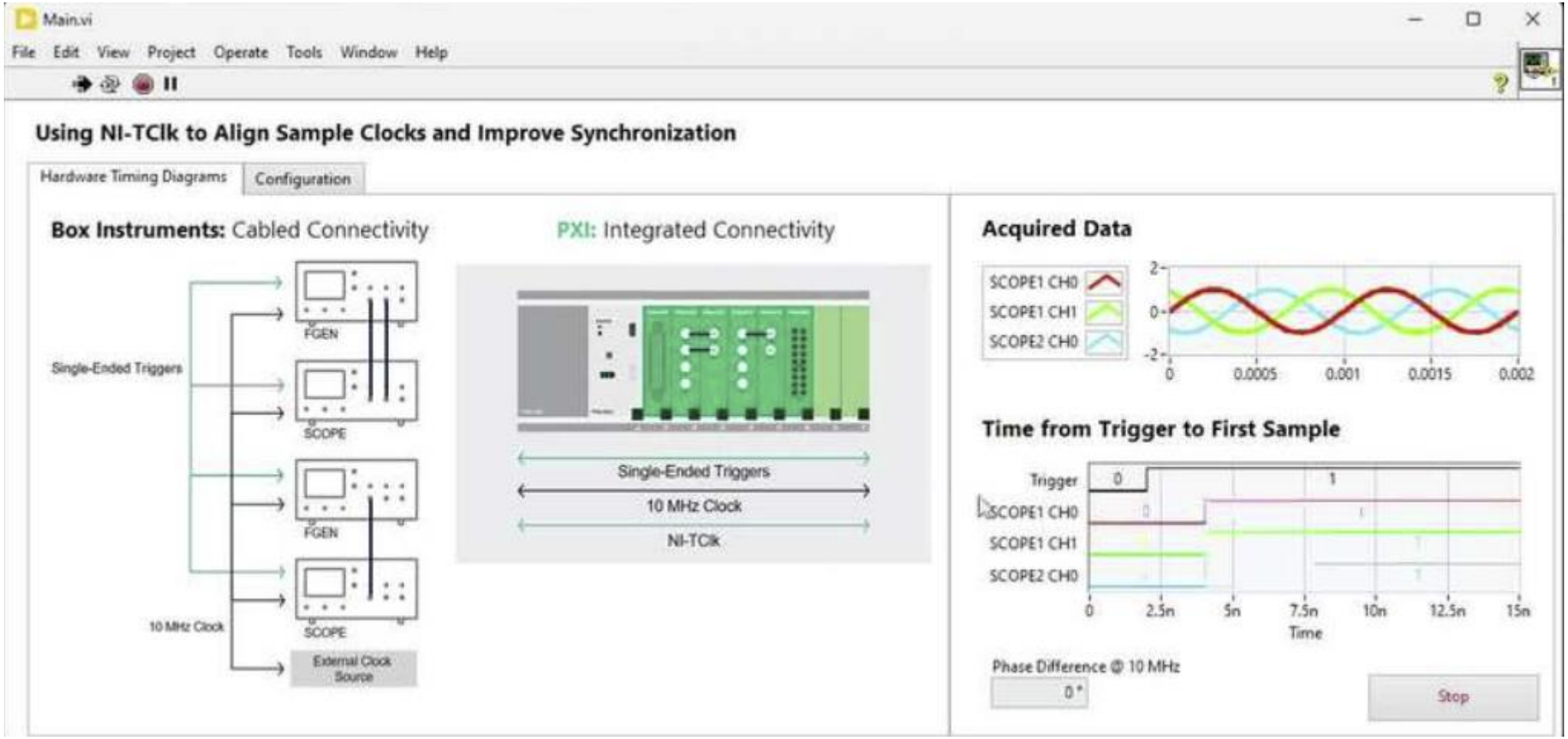
Role of NI-TClk in Programming Flow

NI-TClk coordinates and aligns device sample clocks and TClks.

Devices must be configured and controlled through their own driver APIs.

Advanced features (e.g., external sample clocks and triggers) may require additional configuration specific to instrument drivers and hardware configuration.

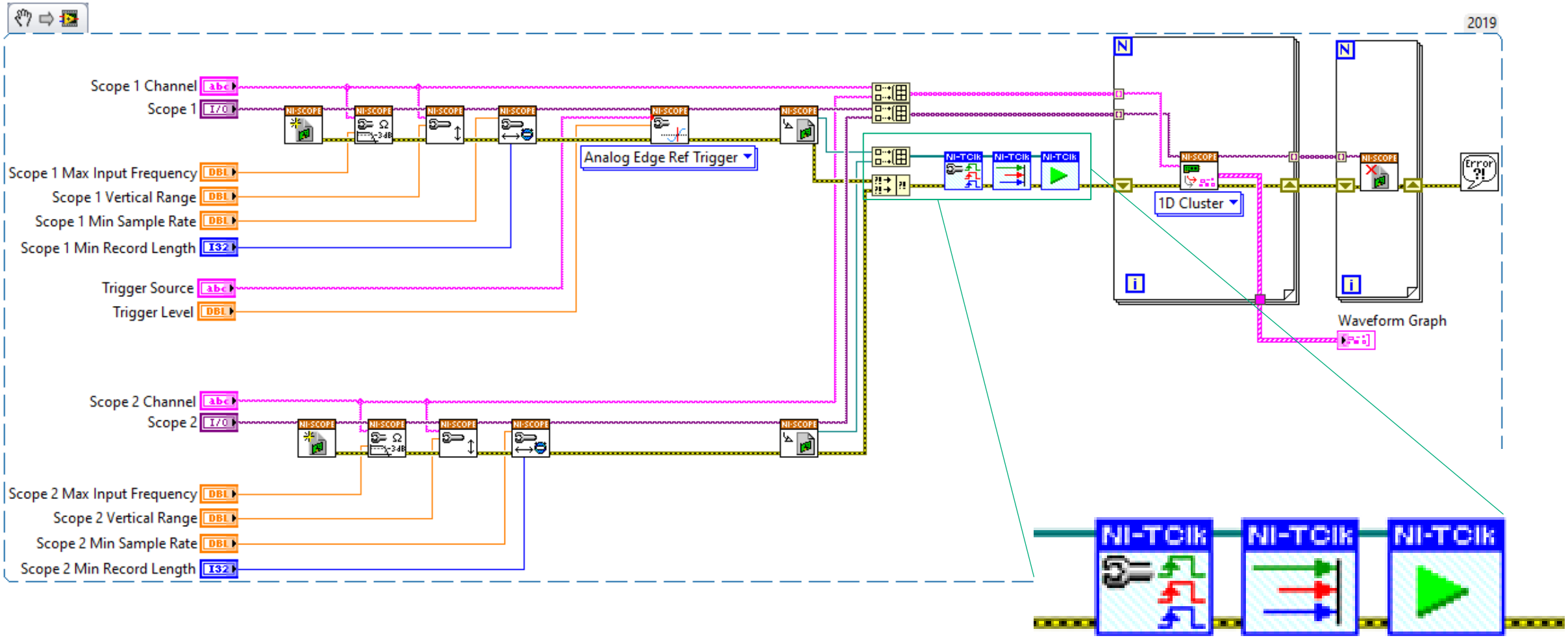
Demo: NI TCLK → FGEN and SCOPE Synchronization



Example Code Source

- https://github.com/eatondan/ni-LunchAndLearn-SignalBasedSynchronizationFundamentalsExamples
  - LabVIEW: Source\MI Sync With TClk
  - Python: Source\MI Sync With TClk Python

Example Screenshot: Single Chassis with Synchronous Triggers



Example Screenshot: Single Chassis with Synchronous Triggers

Set Device References

```
# ---- Configure these for your bench ----
FGEN_RESOURCES  = ["FGEN1", "FGEN2"]   # e.g., "PXI1Slot2" or "Dev1" in NI-MAX
SCOPE_RESOURCES = ["SCOPE1", "SCOPE2"]  # e.g., "PXI1Slot3" or "Dev2"
```

Initialize Device References

Create TCLK Ref List

```
19  # Initiate session refs
20  fgens = [nifgen.Session(FGEN_RESOURCE) for FGEN_RESOURCE in FGEN_RESOURCES]
21  scopes = [niscope.Session(SCOPE_RESOURCE) for SCOPE_RESOURCE in SCOPE_RESOURCES]
22
23  rel_initial_x_list = []          # list for calculating phase
24  hardware_session_list = []       # list for TCLK Sync
```

Example Screenshot: Single Chassis with Synchronous Triggers

Configure Each FGEN

Add FGEN to TCLK Ref List

```
for fgen in fgens:
    # Output a simple standard sine for the demo (FUNC mode)
    fgen.output_mode = nifgen.OutputMode.FUNC   # Select "standard function" mode
    fgen.channels[CHANNEL].configure_standard_waveform(
        waveform=nifgen.Waveform.SINE,
        amplitude=AMP_VPP,           # V pk-pk
        frequency=FREQ_HZ,           # Hz
        dc_offset=0.0,               # V
        start_phase=0.0              # degrees
    )  # Basic usage mirrors NI docs. [1](https://nimi-python.readthedocs.io/en/ma

    # Disable then enable output around initiate for clean start
    fgen.channels[CHANNEL].output_enabled = False

    # Append fgen session reference to TCLK Sync list.
    # Replaces Triggering Setup.
    hardware_session_list.append(fgen)
```

Example Screenshot: Single Chassis with Synchronous Triggers

Configure Each Scope

Add Scope to TCLK Ref List

```python
for scope in scopes:
    # Vertical path on channel 0: DC coupling, requested range V_RANGE
    scope.channels[CHANNEL].configure_vertical(
        range=V_RANGE,
        coupling=niscope.VerticalCoupling.DC,
        offset=0.0,
        probe_attenuation=1.0,
        enabled=True
    )  # Signature & usage per NI SCOPE API examples. [7](https://niscop .readthedocs.io/en/latest/examples.html)[2](https://nimi-pyth

    # Horizontal timing (sample rate, record length, reference position)
    scope.configure_horizontal_timing(
        min_sample_rate=SAMPLE_RATE,
        min_num_pts=NUM_SAMPLES,
        ref_position=0,       # 50% pre/post trigger balance
        num_records=1,
        enforce_realtime=True
    )  # Parameters as in NI-SCOPE docs/examples. [7](https://niscope.re dthedocs.io/en/latest/examples.html)[8](https://www.ni.com/do

    # Set up all the triggers
    if scopes.index(scope) == (len(SCOPE_RESOURCES)-1):
        #Last scope in the list - configure analog edge trigger. No addi ional triggers are needed because of TCLK.
        scope.configure_trigger_edge(
            trigger_source=CHANNEL,
            level=0.0,
            trigger_coupling=niscope.TriggerCoupling.DC,
            slope=niscope.enums.TriggerSlope.POSITIVE
        )  # Edge trigger API & enum documented in NI-SCOPE reference. [ ](https://nimi-python.readthedocs.io/en/master/niscope.html)

    # Append scope session reference to TCLK Sync list.
    # Replaces Triggering Setup.
    # Scopes and fGENs in same list
    hardware_session_list.append(scope)
```

Example Screenshot: Single Chassis with Synchronous Triggers

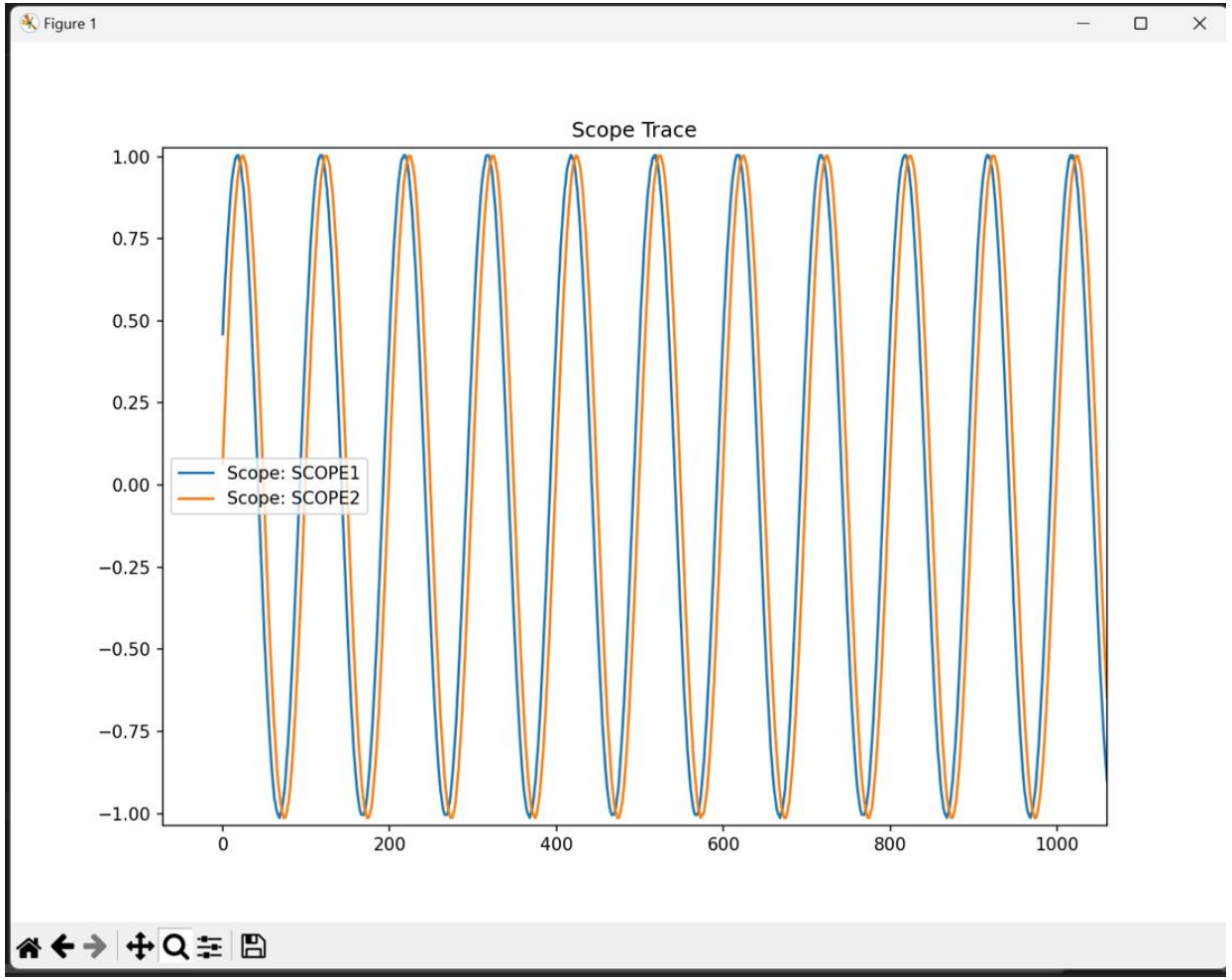
Configure TCLK
Initiate TCLK Synchronization

Initiate/Start TCLK

```
88    # Initiate the TCLK for list
89    # Trigger routing, delay calculations, and resulting synchronization happen automatically
90    nitclk.configure_for_homogeneous_triggers(hardware_session_list)
91    nitclk.synchronize(hardware_session_list, min_tclk_period: 200e-9)
92
93    for fgen in fgens:
94        # Allow fGENs to output
95        fgen.channels[CHANNEL].output_enabled = True
96
97    # Initiate the generation & acquisition together
      nitclk.initiate(hardware_session_list)
```

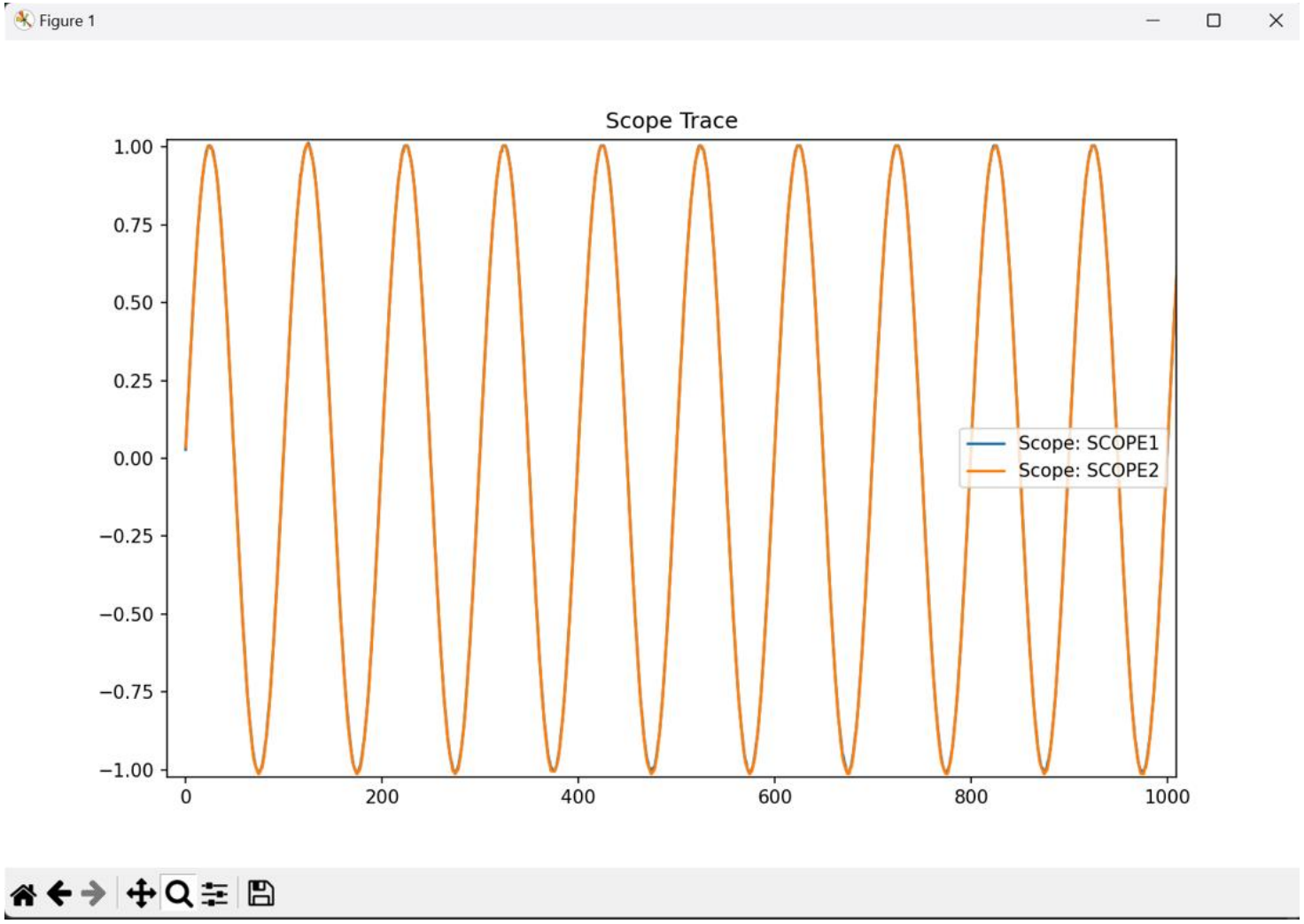
Example Screenshot: Single Chassis with Synchronous Triggers

- NO TCLK: Large Phase (10 to 30 Degrees @ 10 MHz)



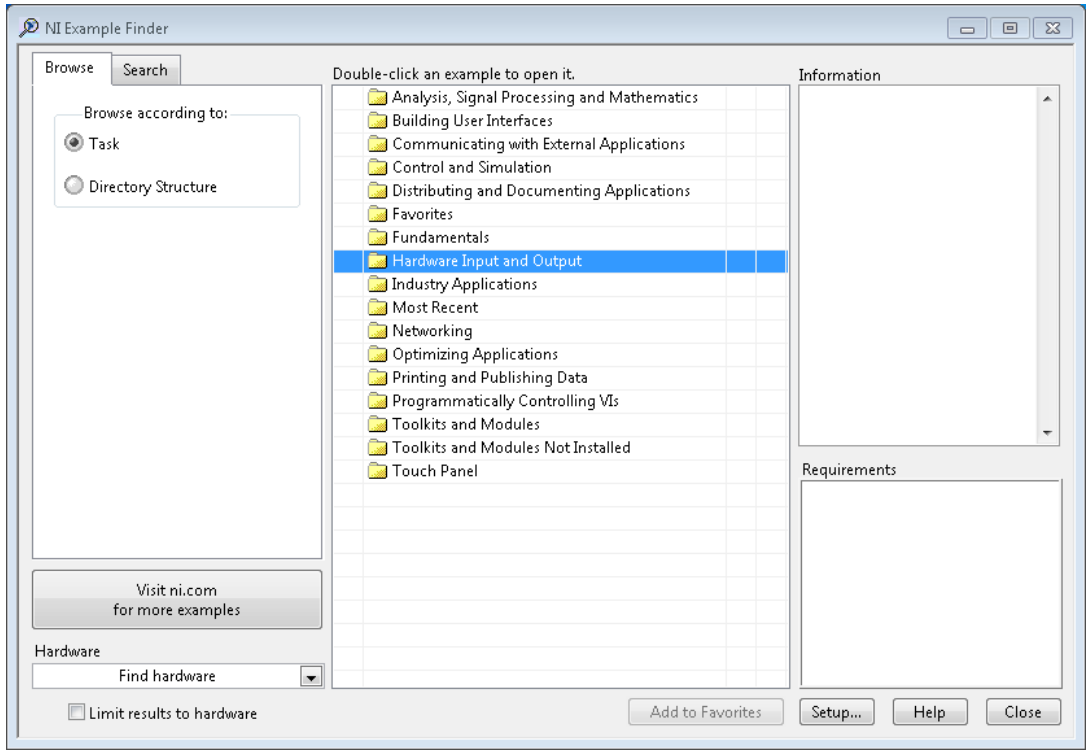
Example Screenshot: Single Chassis with Synchronous Triggers

- TCLK: No Phase (0 Degrees @ 10 MHz)



Additional TCLK Resources

- In-depth Tutorial: Link
- Calibrating NI TCLK (Getting to 10s of picoseconds): Link
- Shipping Examples:



Additional Synchronization Resources

- Training: Link to Purchase
- Training: Link to OnDemand

NI Timing And Sync Examples

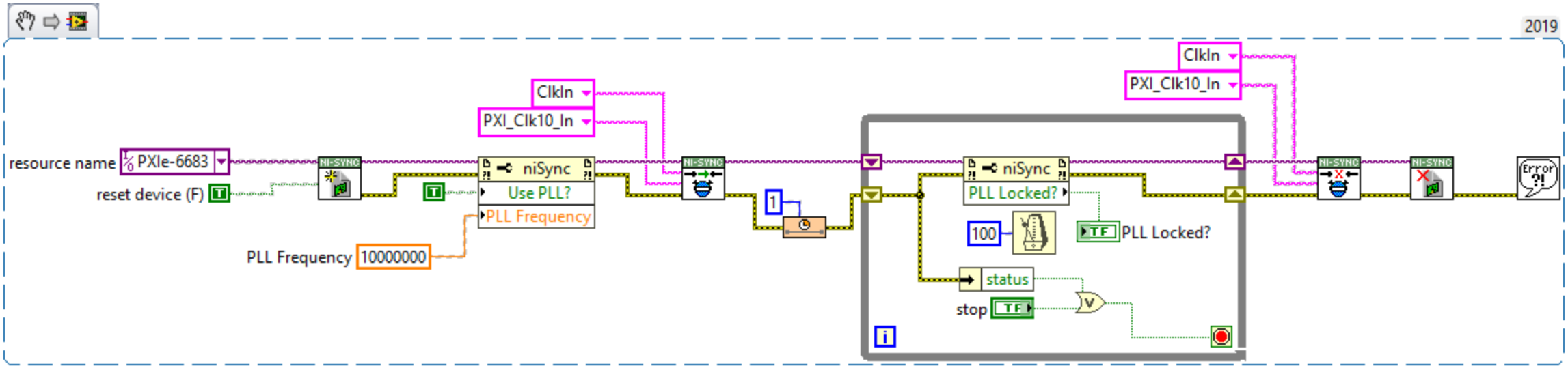
See Shipping Examples

NI Sync Core Functions

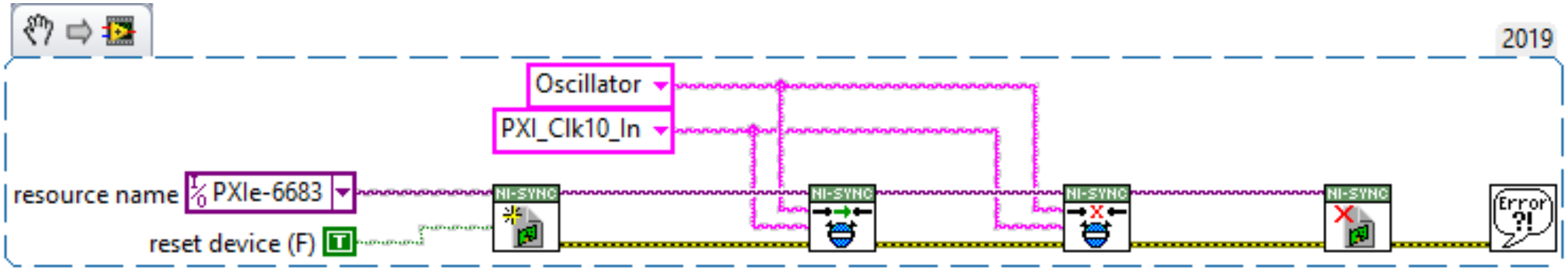
- Initialize
- Connect
- Disconnect
- Close

```python
import nisync
from nisync.constants import PXI_CLK10_IN, OSCILLATOR

# Create a new session with the nisync library
with nisync.Session(resource_name="PXI1Slot10") as session:
    # Connect the onboard oscillator to the PXI_CLK10_IN terminal
    # This setup is typically used for synchronizing measurements across devices
    session.connect_clock_terminals(OSCILLATOR, PXI_CLK10_IN)

    # Perform measurements or operations that require synchronization

    # Once operations are complete, disconnect the terminals
    session.disconnect_clock_terminals(OSCILLATOR, PXI_CLK10_IN)
```

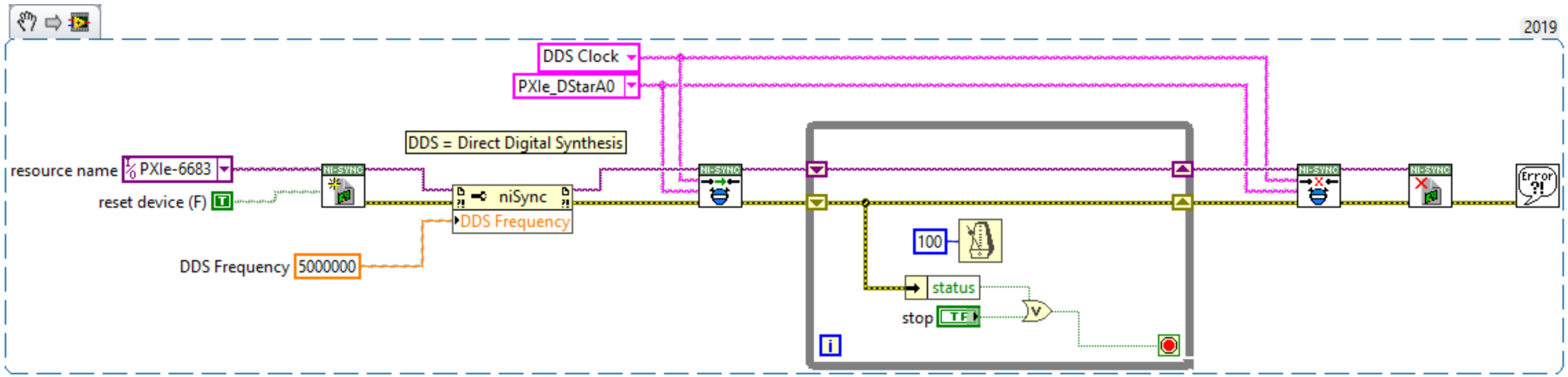
Example Screenshot: Importing External Clock → PXI Clock 10 w/ PLL



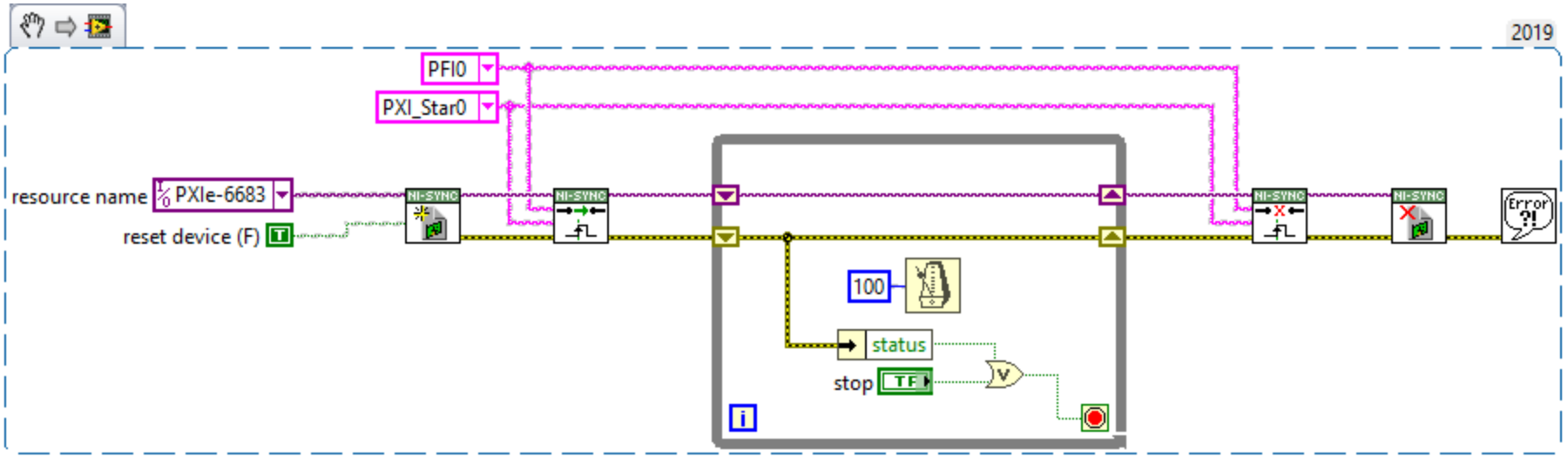
Example Screenshot: Overriding PXI Chassis Clock w/ Sync Modules Clock



Example Screenshot: Creating Custom Clock on Sync Module and Sharing over Star Lines



Example Screenshot: Routing an External Trigger to Star Lines



TSN Appendix

TSN Enabled cRIO, cDAQ, & FieldDAQ

If 1 microsecond is enough… NI TSN Enabled Devices…

TSN Synchronizes multiple devices with no programming required

Formalized as IEEE 802.1 AS (an IEEE 1588 profile)

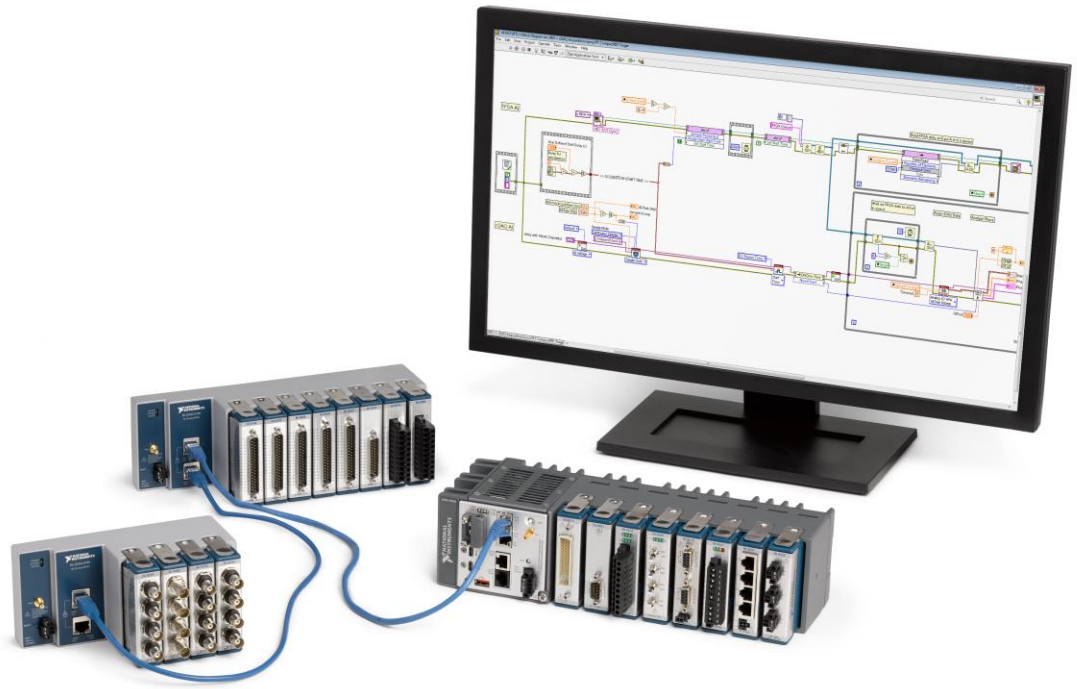
Synchronized networked systems within 1 µs of network time

Synchronize distributed measurements within 1 µs

- Same start time
- No drift over time

How: Plug in ethernet cables…

Supported Devices: TSN Enabled cRIO, cDAQ, & FieldDAQ

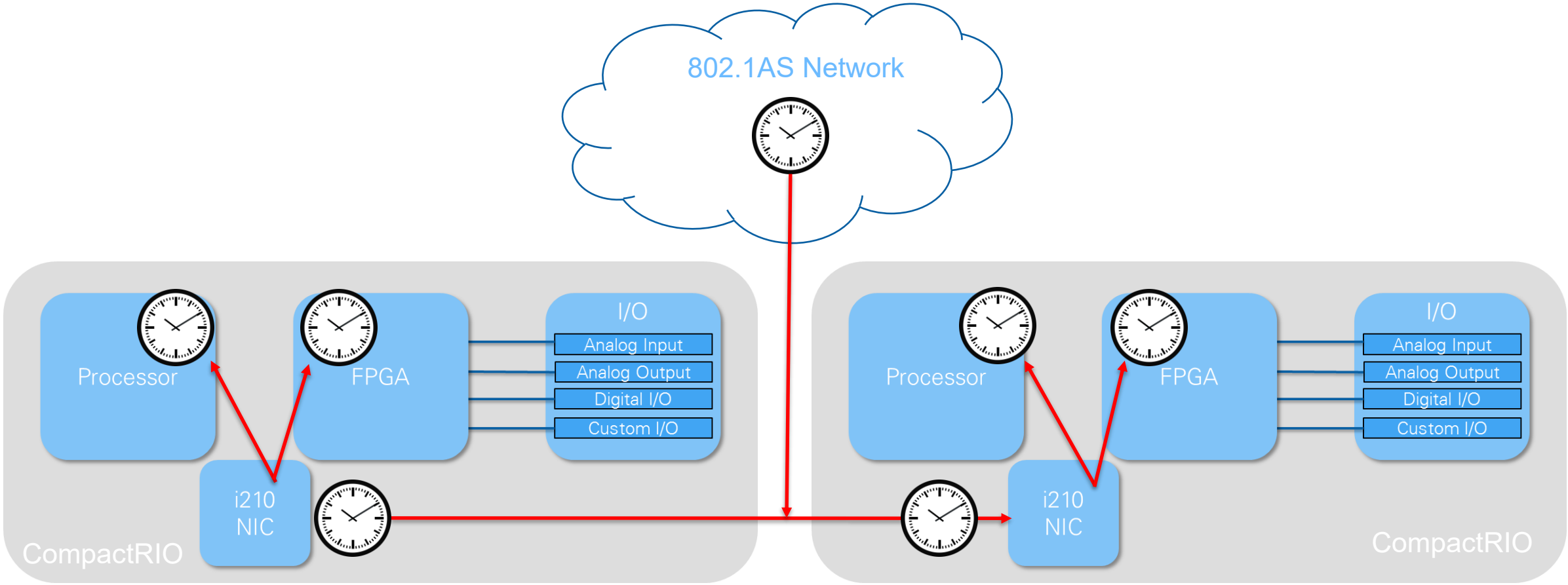


CALLBACK COMING…

EMERSON

IEEE 802.1AS on CompactRIO And Industrial Controllers

Time Synchronization



All CPUs and FPGAs agree on time (enables start time synchronization)

Local clocks are disciplined (eliminates drift)

EMERSON

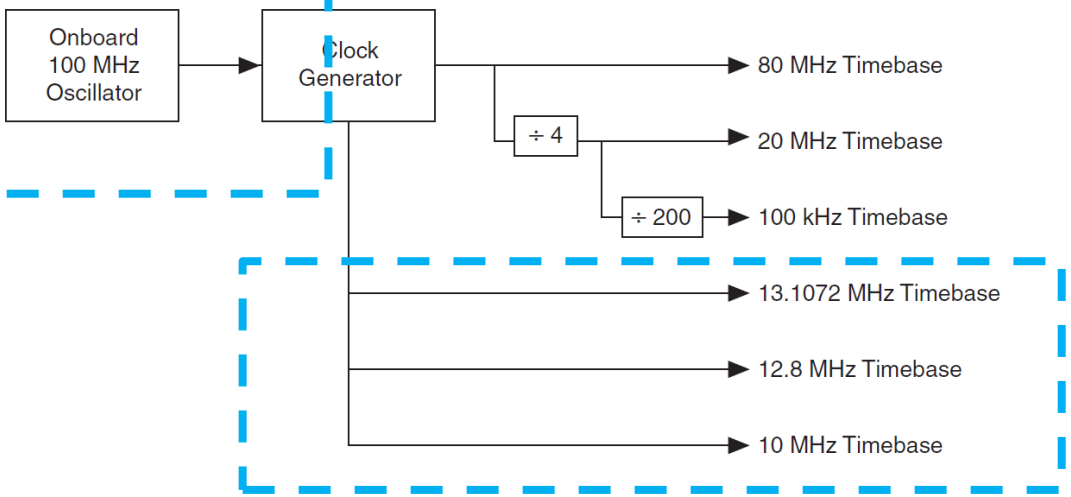
Chassis Clock Generation & AI Timing Engines

Chassis Clock Generation

Network Time

Adjusts

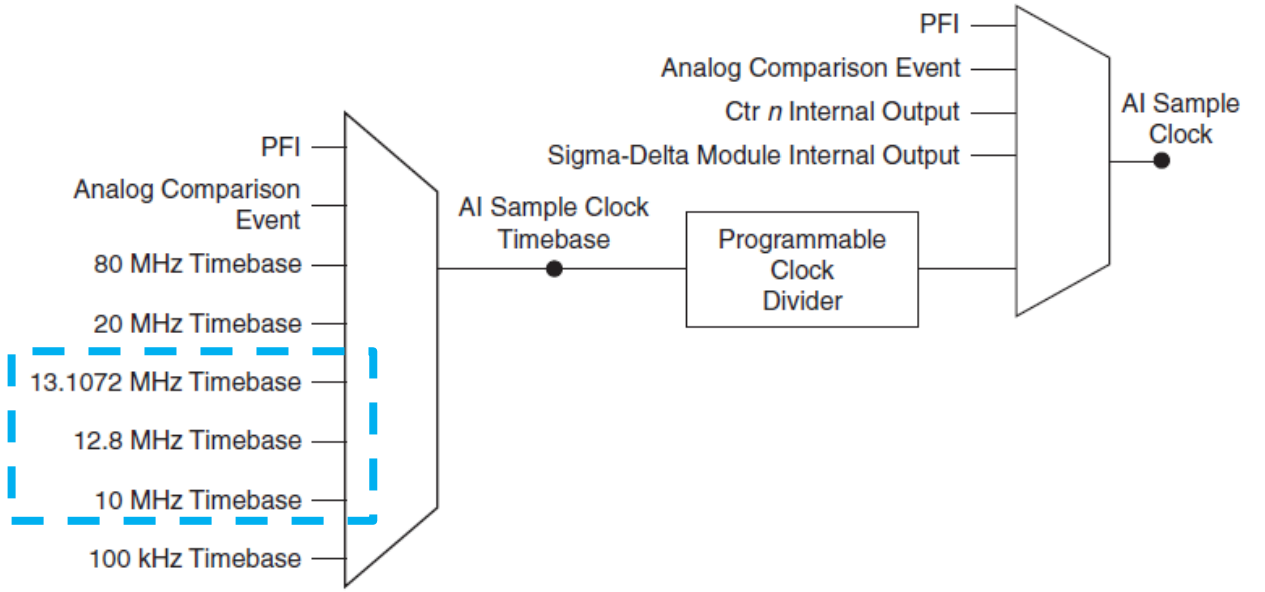
Figure 7-1. Clock Routing Circuitry



Only on TSN Chassis

AI Timing Engines

Figure 3-1. AI Sample Clock Timing Options

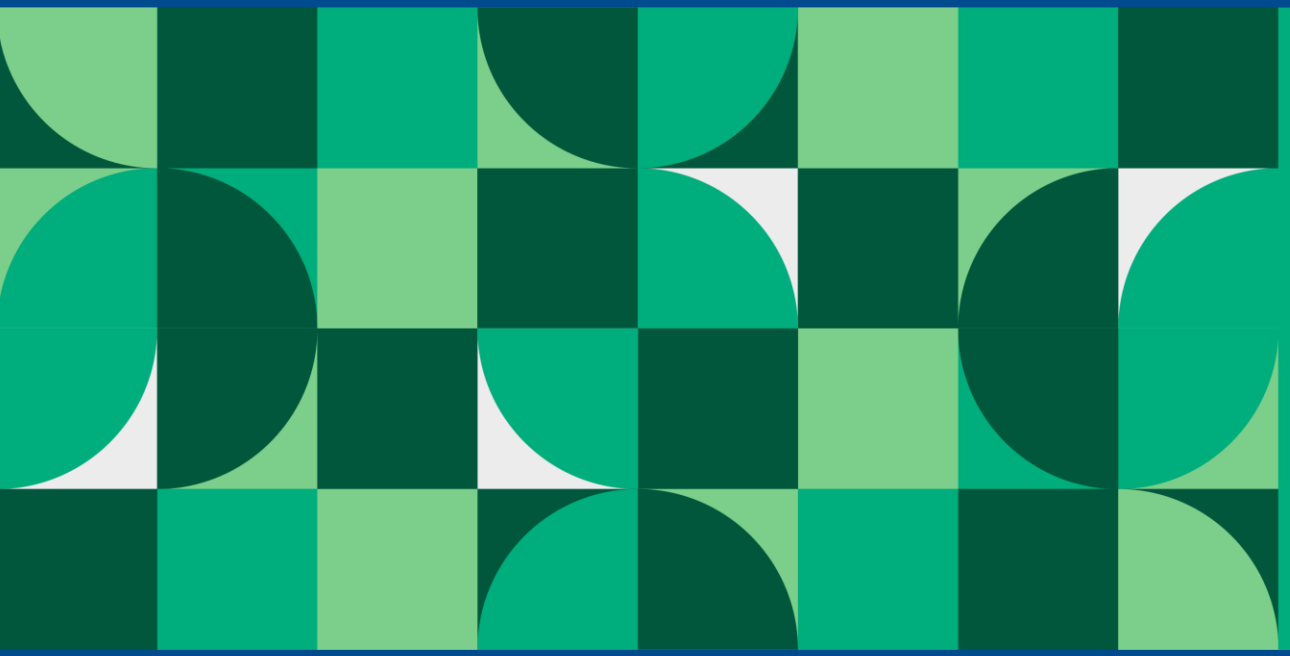


Questions?

Daniel Eaton
Chief Field Applications Engineer
CLA, CLED, 20 years “doing” w/ NI
Daniel.eaton@emerson.com

We are here to help!

- Pull us in to design conversations
- Let us help de-risk projects
- Tell us what you want to learn about next



NI is now part of Emerson.